



SYRIAN PRIVATE UNIVERSITY

**FACULTY OF ARTIFICIAL INTELLIGENCE
ENGINEERING**

DEPARTMENT OF INTELLIGENT SOFTWARE ENGINEERING

SMART HEALTH ASSISTANT

Prepared by: May Al-Zoubi & Malek Essa

Supervised by : Eng. Shadi Blaidi & Eng. Maya Al-Ahmad

Academic Year: 2025-2026

Damascus, Syria

Abstract

The current era is witnessing a rapid acceleration in the reliance on digital technologies and smart systems to improve the quality of life. In the academic environment, students often face study pressures that lead to neglecting essential health aspects, such as regular sleep schedules and adequate water intake, which negatively impacts their focus and overall performance.

This project aims to design and develop an integrated smart health assistant that acts as a personal tool to track and improve users' health habits. The proposed software system relies on tracking daily inputs (such as sleep hours and fluids consumed) and analyzing them to provide customized recommendations and alerts that help build a sustainable healthy routine.

Through this project, we seek to provide a technical solution that combines a user-friendly interface with precision in data processing. This contributes to bridging the gap between technology and preventive health, which positively reflects on health awareness and increases academic and daily productivity.

Keywords: Smart Health Assistant, Digital Health, Habit Tracking, Sleep Quality, Water Consumption.

<u>Abstract (English).....</u>	<u>I</u>
--------------------------------	----------

<u>Table of Contents.....</u>	<u>II</u>
-------------------------------	-----------

<u>List of Figures.....</u>	<u>III</u>
-----------------------------	------------

<u>List of Tables.....</u>	<u>IV</u>
----------------------------	-----------

<u>Glossary of Terms.....</u>	<u>V</u>
-------------------------------	----------

<u>Part 1: Project Introduction.....</u>	<u>15</u>
--	-----------

<u>Chapter 1: General Introduction.....</u>	<u>15</u>
---	-----------

<u>Project Overview and 1.1 Background.....</u>	<u>15</u>
---	-----------

<u>Problem 2.1 Statement.....</u>	<u>15</u>
-----------------------------------	-----------

<u>Project 3.1 Objectives.....</u>	<u>16</u>
------------------------------------	-----------

<u>Project Scope, Constraints, and 4.1 Assumptions.....</u>	<u>16</u>
---	-----------

<u>Project 5.1 Contribution.....</u>	<u>17</u>
--------------------------------------	-----------

<u>Research Methodology and Scientific</u>	<u>6.1</u>
<u>Methods.....</u>	<u>17</u>
<u>Document</u>	<u>7.1</u>
<u>Structure.....</u>	<u>17</u>
<u>Definitions and Basic</u>	<u>8.1</u>
<u>Concepts.....</u>	<u>18</u>
<u>Chapter 2: Methodology and Project</u>	
<u>Management.....</u>	<u>19</u>
<u>Selected Development</u>	<u>1.2</u>
<u>Methodology.....</u>	<u>19</u>
<u>Project Timeline (Gantt</u>	<u>2.2</u>
<u>Chart).....</u>	<u>19</u>
<u>Risk</u>	<u>3.2</u>
<u>Management.....</u>	<u>19&20</u>
<u>Team Roles and Task</u>	<u>4.2</u>
<u>Distribution.....</u>	<u>20</u>
<u>Project Quality</u>	<u>5.2</u>
<u>Management.....</u>	<u>20</u>
<u>Project Management</u>	<u>6.2</u>
<u>Tools.....</u>	<u>20</u>
<u>Monitoring and Continuous</u>	<u>7.2</u>
<u>Evaluation.....</u>	<u>21</u>
<u>Software Configuration Management</u>	<u>8.2</u>
<u>(SCM).....</u>	<u>21</u>

<u>Part 2: Theoretical Study and</u>	
<u>Analysis.....</u>	<u>22</u>
<u>Chapter 3: Literature Review and Previous</u>	
<u>Studies.....</u>	<u>22</u>
<u>Previous Studies in Smart Health 1.3</u>	
<u>Systems.....</u>	<u>22</u>
<u>Modern Technologies and 2.3</u>	
<u>Frameworks.....</u>	<u>22&23</u>
<u>Comparative 3.3</u>	
<u>Analysis.....</u>	<u>23</u>
<u>Research Gap and 4.3</u>	
<u>Innovation.....</u>	<u>23</u>
<u>Theoretical Framework and Computational 5.3</u>	
<u>Models.....</u>	<u>23</u>
<u>Modern and Future Trends in the 6.3</u>	
<u>Field.....</u>	<u>24</u>
<u>Chapter 4: Requirements Modeling and</u>	
<u>Analysis.....</u>	<u>25</u>
<u>Feasibility 1.4</u>	
<u>Study.....</u>	<u>25</u>
<u>Requirements 2.4</u>	
<u>Elicitation.....</u>	<u>25</u>
<u>Stakeholders 3.4</u>	
<u>Identification.....</u>	<u>25&26</u>

<u>Functional Requirements 4.4</u>	
<u>(FR).....</u>	<u>26</u>
<u>Non-Functional Requirements 5.4</u>	
<u>(NFR).....</u>	<u>26&27</u>
<u>Requirements Prioritization 6.4</u>	
<u>(MoSCOW).....</u>	<u>27</u>
<u>UML 7.4</u>	
<u>Modeling.....</u>	<u>27</u>
<u>Use Case 1.7.4</u>	
<u>Diagram.....</u>	<u>28</u>
<u>Use Case 2.7.4</u>	
<u>Description.....</u>	<u>29</u>
<u>Activity 3.7.4</u>	
<u>Diagram.....</u>	<u>42</u>
<u>Sequence 4.7.4</u>	
<u>Diagram.....</u>	<u>43</u>
<u>Test Case Descriptions 5.7.4</u>	
<u>Overview.....</u>	<u>44</u>
<u>Modeling Based on Traditional Methodology 8.4</u>	
<u>(Waterfall).....</u>	<u>44</u>
<u>Software Requirements Specification 1.8.4</u>	
<u>(SRS).....</u>	<u>44</u>
<u>Requirements Analysis and 9.4</u>	
<u>Documentation.....</u>	<u>44&45</u>

<u>Requirements Traceability Matrix 1.9.4</u>	
<u>(RTM).....</u>	<u>44&45</u>
<u>Initial Test 2.9.4</u>	
<u>Cases.....</u>	<u>45</u>
<u>Part 3: Design and</u>	
<u>Architecture.....</u>	<u>46</u>
<u>Chapter 5: System Architectural</u>	
<u>Design.....</u>	<u>46</u>
<u>High-Level 1.5</u>	
<u>Design.....</u>	<u>46</u>
<u>System Block 1.1.5</u>	
<u>Diagram.....</u>	<u>47</u>
<u>Architectural Design 2.1.5</u>	
<u>Pattern.....</u>	<u>47&48</u>
<u>Architectural Design (Class 3.1.5</u>	
<u>Diagram).....</u>	<u>49</u>
<u>Database 2.5</u>	
<u>Design.....</u>	<u>49</u>
<u>Entity-Relationship Diagram 1.2.5</u>	
<u>(ERD).....</u>	<u>49</u>
<u>Relational 2.2.5</u>	
<u>Schema.....</u>	<u>50</u>
<u>Database 3.2.5</u>	
<u>Constraints.....</u>	<u>50</u>

<u>External APIs</u>	<u>3.5</u>
<u>Design</u>	<u>51</u>
<u>Security and Authorization</u>	<u>4.5</u>
<u>Design</u>	<u>51</u>
<u>Part 4: Implementation and</u>	
<u>Testing</u>	<u>52</u>
<u>Chapter 6: Implementation Environment and</u>	
<u>Tools</u>	<u>52</u>
<u>Software Tools and</u>	<u>1.6</u>
<u>Utilities</u>	<u>52</u>
<u>Development Environment and Project</u>	<u>2.6</u>
<u>Structure</u>	<u>52</u>
<u>Programming Languages and</u>	<u>3.6</u>
<u>Frameworks</u>	<u>52&53</u>
<u>Database Server and Operating</u>	<u>4.6</u>
<u>System</u>	<u>53</u>
<u>Libraries and</u>	<u>5.6</u>
<u>Dependencies</u>	<u>53</u>
<u>Version Control and Source Code</u>	<u>6.6</u>
<u>Management</u>	<u>53</u>
<u>Continuous Integration and Continuous</u>	<u>7.6</u>
<u>Deployment (CI/CD)</u>	<u>53&54</u>
<u>Testing</u>	<u>8.6</u>
<u>Environment</u>	<u>54</u>

<u>Chapter 7: Software</u>	
<u>Implementation.....</u>	<u>54</u>
<u>Project File 1.7</u>	
<u>Structure.....</u>	<u>54&55&56</u>
<u>Implementation of 2.7</u>	
<u>Modules.....</u>	<u>56</u>
<u>APIs Implementation 3.7</u>	
<u>(Controllers).....</u>	<u>56</u>
<u>Data Access Layer Implementation 4.7</u>	
<u>(Repositories).....</u>	<u>56</u>
<u>Business Logic Layer Implementation 5.7</u>	
<u>(Services).....</u>	<u>57</u>
<u>UI 6.7</u>	
<u>Implementation.....</u>	<u>57</u>
<u>Intelligent Algorithms 7.7</u>	
<u>Implementation.....</u>	<u>58</u>
<u>Chapter 8: Testing and</u>	
<u>Evaluation.....</u>	<u>59</u>
<u>Test 1.8</u>	
<u>Plan.....</u>	<u>59</u>
<u>Types of 2.8</u>	
<u>Testing.....</u>	<u>59</u>
<u>Test 3.8</u>	
<u>Diagrams.....</u>	<u>60</u>

<u>Test Case 1.3.8</u>	
<u>Diagrams.....</u>	<u>60</u>
<u>Test Results 2.3.8</u>	
<u>Diagrams.....</u>	<u>61</u>
<u>Test Results and 4.8</u>	
<u>Analysis.....</u>	<u>61&62</u>
<u>Quality 5.8</u>	
<u>Metrics.....</u>	<u>62</u>
<u>Usability 6.8</u>	
<u>Testing.....</u>	<u>62</u>
<u>Part 5: Results and</u>	
<u>Recommendations.....</u>	<u>63</u>
<u>Chapter 9: Results and</u>	
<u>Analysis.....</u>	<u>63</u>
<u>System Application 1.9</u>	
<u>Results.....</u>	<u>63</u>
<u>Comparative Analysis with Existing 2.9</u>	
<u>Systems.....</u>	<u>64&65</u>
<u>Meeting 3.9</u>	
<u>Objectives.....</u>	<u>65</u>
<u>Strengths and Weaknesses 4.9</u>	
<u>Analysis.....</u>	<u>65</u>
<u>Key Performance Indicators 5.9</u>	
<u>(KPIs).....</u>	<u>66</u>

**Chapter 10: Conclusion and
Recommendations.....67**

**Project Summary and Key 1.10
Achievements.....67**

**10.2 Difficulties and Challenges
Faced.....67**

Proposals for Future Developmen 3.10

.....67

Recommendations 4.10

.....68

Part 6:Appendices & References

.....69.➡79

List of Figures

Figure 1.1: Smart Health Assistant Logo/Branding

Figure 2.1: Project Timeline (Gantt Chart)

Figure 4.1: Main Use Case Diagram

Figure 4.2: Activity Diagram for Logging Water Intake

Figure 4.3: Sequence Diagram for Data Synchronization

Figure 5.1: High-Level System Architecture

Figure 5.2: Class Diagram

Figure 5.3: Entity-Relationship Diagram (ERD)

Figure 9.1: Application Main Dashboard Screenshot

Figure 9.2: Water Tracking and Fasting Mode UI

List of Tables

Table 2.1: Risk Management and Mitigation Plan

Table 3.1: Comparative Analysis of Existing Health Systems

Table 4.1: Detailed Use Case Description - UC-01

Table 6.1: Development Tools and Frameworks

Table 8.1: System Test Cases and Results

Glossary of Terms

Term / Acronym	Definition
UI (User Interface)	The visual elements, screens, and buttons of the application through which a user interacts.
UX (User Experience)	The overall experience and satisfaction of the user when using the application, focusing on ease of use.
API (Application Programming Interface)	A set of rules and protocols that allows the mobile frontend to communicate with the backend server.
Waterfall Methodology	A linear sequential design approach for software development, where progress flows downwards through phases like requirements, design, implementation, and testing.
Hydration Factor	A mathematical multiplier used to calculate the actual net water gained from different beverages (e.g., coffee vs. pure water).
Fasting Mode	A customized application feature that suppresses daytime water and health notifications during fasting periods like Ramadan.
ERD (Entity-Relationship Diagram)	A graphical representation of the database structure, showing entities and their relationships.

Part 1: Project Introduction

Chapter 1: General Introduction

1.1 Project Overview and Background

In light of rapid technological development, smartphones and mobile devices have become an integral part of our daily lives, and their role is no longer limited to communication only, but extends to managing and organizing lifestyles. From here emerged the importance of "Digital Health" technologies, which aim to employ technology to improve the quality of life and prevent diseases.

Developing smart health assistant systems is an essential step in this field, as these systems monitor and analyze daily behaviors to provide proactive guidance. This project focuses on building a "**Smart Health Assistant**" dedicated to tracking and improving basic daily habits, with a primary focus on sleep quality and water consumption rates, due to their direct impact on mental and physical activity.

1.2 Problem Statement

Many individuals, especially in academic and work environments filled with pressure, suffer from difficulty maintaining a balanced healthy routine. The core problem lies in neglecting vital habits such as drinking sufficient amounts of water and getting regular sleep hours, which leads to exhaustion, poor concentration, and decreased productivity.

Although there are many health applications in the market, many of them suffer either from complexity and too many details that distract the user, or from a lack of flexibility and customization that suits individual needs, creating a gap and an urgent need for a smart, simple, and integrated system.

1.3 Project Objectives

The primary goal of this project is to design and develop a "**Smart Health Assistant**" application that enhances health awareness and fosters positive habit formation among users. The specific objectives are as follows:

- **Tracking Vital Habits:** Providing a user-friendly interface to record and monitor daily sleep duration and water consumption.
- **Smart Analysis and Alerting:** Processing user data to deliver customized notifications and reminders at appropriate times, helping users adhere to their healthy routines.
- **Sustainable Motivation:** Designing a User Experience (UX) that encourages long-term consistency and reduces the likelihood of abandoning the app.
- **Periodic Reporting:** Displaying weekly or monthly summaries that illustrate the user's progress and the impact of their commitment on overall well-being.

1.4 Project Scope, Constraints, and Assumptions

- **Scope:** The project covers the development of a mobile application with a dedicated backend to manage user profiles, track water and sleep data, and provide intelligent notification logic.
- **Constraints:** The development is limited to the academic semester timeline (ending in April 2026). It also relies on the availability of standard smartphone sensors and consistent internet for cloud syncing.
- **Assumptions:** It is assumed that users possess basic smartphone literacy and that improving tracking habits will lead to better health outcomes for the target audience.

1.5 Project Contribution

This project contributes to the field of "Digital Health" by offering a simplified, student-focused tool that avoids the clutter of comprehensive systems. It introduces a specific "**Fasting Mode**" to handle regional cultural needs and uses intelligent algorithms to adapt health reminders to the user's specific daily schedule.

1.6 Research Methodology and Scientific Methods

This project adopts an **Applied Engineering and Experimental Methodology**. The research process is structured around the following scientific steps:

- **Descriptive and Analytical Phase:** Reviewing existing literature and analyzing current health tracking applications to identify specific gaps (e.g., the lack of contextual smart notifications and fasting modes).
- **Design Phase:** Formulating the system architecture and database models required to process user health data (sleep and water intake) effectively.
- **Experimental and Implementation Phase:** Developing the mobile application and backend logic, applying specific algorithms for dynamic hydration calculation and notification scheduling.
- **Evaluation Phase:** Conducting software testing (Functional and Non-Functional) to measure system reliability, responsiveness, and user experience to ensure the predefined project objectives are met.

1.7 Document Structure

The thesis is organized as follows:

- **Chapter 2** discusses the project management and the Agile development methodology used.
- **Chapter 3** reviews literature and compares the Smart Health Assistant with existing market solutions.
- **Chapter 4** details the requirements modeling using UML diagrams.
- **Chapter 5** covers the system architecture and database design.

- **Chapters 6 through 8** focus on the implementation environment, coding, and testing phases.
- **Chapters 9 and 10** present the final results, conclusions, and future work.

1.8 Definitions and Basic Concepts

The following list outlines the key technical and domain-specific terms used throughout this document:

- **Digital Health:** The intersection of technology and healthcare aimed at improving everyday lifestyle and well-being.
- **Waterfall Methodology:** A linear sequential project management approach where each phase of the development lifecycle must be fully completed before the next phase begins.
- **Hydration Factor:** A calculated multiplier that adjusts the recorded net water intake based on the type of beverage consumed (e.g., adjusting for caffeine).
- **Fasting Mode:** A specialized system feature that pauses daytime health notifications to accommodate cultural or religious fasting schedules like Ramadan.
- **REST API:** An architectural style for an application programming interface that allows the mobile frontend to securely communicate with the backend server.
- **Flutter:** An open-source UI software development kit created by Google, utilized in this project to build the cross-platform mobile application.

To ensure the project's success, potential risks were identified and mitigation strategies were developed as shown in the table below:

Risk	Impact	Mitigation Strategy
Technical Integration Issues (FE/BE)	High	Regular API testing using Postman before full integration.
Data Synchronization Failures	Medium	Implementing a local SQLite cache to prevent data loss.

2.4 Team Roles and Task Distribution

The project was a collaborative effort between two members, with clearly defined roles to ensure efficiency:

- **May Al-Zoubi:** Frontend Developer and UI/UX Designer. Responsible for creating the mobile interfaces using Flutter, managing user experience flows, and integrating client-side logic.
- **Malek Essa:** Backend Developer and Database Administrator. Responsible for developing the Python-based REST API, designing the system logic, and managing the MySQL database.
- **Shared Responsibilities:** Both members participated in requirements gathering, architectural planning, and system-wide testing.

2.5 Project Quality Management

To ensure the delivery of a high-quality product, several quality assurance measures were integrated into the development lifecycle:

- **Code Reviews:** Regular peer reviews between team members before merging new features.
- **Automated & Manual Testing:** Using unit tests for critical backend logic (such as hydration factor calculations) and manual testing for UI/UX flows on different screen sizes.
- **API Testing:** Utilizing Postman to validate all REST endpoints before mobile integration.

2.6 Project Management Tools

The team utilized various tools to organize tasks, manage source code, and track progress effectively:

- **Trello:** Used to track tasks across the different sequential phases of the project (Analysis, Design, Coding, Testing).
- **GitHub:** Used for version control and collaborative code management.

- **Google Workspace:** Docs and Sheets were used for documentation, Gantt chart creation, and progress reporting.

2.7 Monitoring and Continuous Evaluation

Project monitoring was maintained through structured communication and evaluation cycles:

- **Weekly Meetings:** Internal team meetings to discuss the progress of the current development phase, challenges, and upcoming tasks.
- **Supervisor Reviews:** Bi-weekly sessions with the project supervisors to evaluate milestones, receive feedback, and align technical progress with academic requirements.

2.8 Software Configuration Management (SCM)

A strict Software Configuration Management plan was established using **Git** to handle code versions safely and avoid conflicts:

- **Branching Strategy:** The repository was structured using clear branches: `main` (for production-ready code), `dev` (for active development and integration), and specific feature branches like `feature/fasting-mode` for individual tasks.
- **Commit Conventions:** Using descriptive commit messages to document the purpose of changes clearly.

Part 2: Theoretical Study and Analysis

Chapter 3: Literature Review and Previous Studies

3.1 Previous Studies in Smart Health Systems

Smart health assistants have evolved significantly, moving from simple tracking tools to integrated systems that offer personalized insights. This section reviews various existing systems to understand their focus and limitations:

- **Sleep Quality Trackers (e.g., Sleep Cycle):** These applications utilize smartphone sensors to analyze sleep stages and provide detailed quality reports. However, they often operate in isolation, failing to link sleep patterns with daytime habits like hydration.
- **Hydration Monitoring Apps (e.g., WaterMinder):** These systems focus on calculating daily water needs and sending reminders. Their main drawback is the use of "blind" notifications that do not adapt to the user's specific schedule, such as academic lectures or fasting periods.
- **Comprehensive Lifestyle Platforms (e.g., Habitica):** These platforms use gamification to motivate habit changes. While effective for motivation, their complex interfaces can overwhelm users who seek a direct and focused tracking experience.

3.2 Modern Technologies and Frameworks

To build a robust and cross-platform solution, the **Smart Health Assistant** leverages the following technologies:

- **Flutter:** For creating a responsive and unified user interface across mobile platforms.
- **Python:** For backend logic, processing hydration algorithms, and managing notification scheduling.
- **MySQL & SQLite:** Used for cloud synchronization and local data persistence respectively, ensuring offline functionality.

3.3 Comparative Analysis

The following table summarizes the comparison between our proposed system and existing solutions:

System Type	Strengths	Weaknesses	Smart Health Assistant Solution
Sleep Trackers	Detailed analysis	No link to water intake	Integrates sleep and water data for holistic insights.
Water Apps	Simple interface	Blind notifications	Smart notifications that adapt to user schedule and fasting.
Comprehensive	High motivation	Complex UX	Simplified, focused UI to lower the learning curve.

3.4 Research Gap and Innovation

Based on the analysis, there is a clear gap for a system that provides a focused, non-distracting experience while intelligently adapting to cultural and academic schedules (e.g., Fasting Mode). Our innovation lies in merging these vital habits with an intelligent notification engine that respects the user's daily context.

3.5 Theoretical Framework and Computational Models

The foundation of the **Smart Health Assistant** relies on established architectural and computational models to ensure accuracy and reliability:

- **RESTful Architecture:** The communication between the Flutter frontend and the Python backend is built upon Representational State Transfer (REST) principles. This ensures stateless, scalable, and standardized data exchange using HTTP methods.
- **Hydration Factor Algorithm:** The system utilizes a mathematical weighting model to calculate net hydration. For instance,

consuming 250ml of coffee does not yield 250ml of pure hydration due to its mild diuretic effect. The system applies a specific multiplier (e.g., 0.8 for caffeinated drinks) to adjust the daily progress dynamically and accurately.

- **Rule-Based Notification Logic:** The intelligent alerting system uses a rule-based engine that evaluates multiple conditions (current time, user's sleep schedule, fasting mode status, and remaining hydration goal) before triggering a push notification.

3.6 Modern and Future Trends in the Field

The landscape of digital health and smart assistants is rapidly evolving. Key trends shaping the future of this domain include:

- **Predictive Analytics and Machine Learning:** Moving beyond simple manual tracking, modern systems are shifting towards predicting user fatigue or dehydration based on historical data patterns.
- **IoT and Wearable Integration:** The seamless collection of physiological data (e.g., heart rate variability, automated sleep phase detection) directly from smartwatches and fitness trackers, eliminating the need for manual user input.
- **Context-Aware Computing:** Systems are becoming highly adaptive to the user's biological and cultural routines. Innovations like our proposed "Fasting Mode" align with the growing demand for highly personalized, context-aware digital therapeutics.

Chapter 4: Requirements Modeling and Analysis

4.1 Feasibility Study

Before initiating the development phase, a feasibility study was conducted to ensure the project's viability:

- **Technical Feasibility:** The system is highly feasible. The required technologies (Flutter for cross-platform mobile UI, Python for the backend logic, and MySQL/SQLite for data management) are well-documented, open-source, and fully capable of handling the project's requirements, including background notifications and local caching.
- **Financial Feasibility:** The project is financially viable for a student initiative. It relies on free, open-source frameworks. The only anticipated costs are related to cloud database hosting and server deployment, which can be initially covered using free-tier cloud services during the academic phase.

4.2 Requirements Elicitation

To gather accurate requirements that reflect actual user needs, the following methods were employed:

- **Questionnaires:** A digital survey was distributed among university students to understand their daily hydration and sleep habits, and to identify the main reasons they abandon traditional health tracking apps.
- **Market Observation:** Reviewing user feedback and complaints on existing applications in the Google Play Store to avoid common pitfalls (e.g., annoying notification schedules).

4.3 Stakeholders Identification

Identifying stakeholders is crucial to align the system's capabilities with user expectations:

- **End-Users (Students and General Public):** They need a simple, fast, and distraction-free interface to log their health data. They

expect the system to respect their time through smart, context-aware notifications.

- **System Administrators:** Require a stable backend system to manage user data securely and monitor server health.
- **Future Developers:** Need a clean, modular codebase (separated frontend and backend) to easily integrate future expansions like diet tracking or medical records.

4.4 Functional Requirements (FR)

The core functionalities the system must perform include:

1. The user must be able to securely log in and create an account.
2. The user must be able to update their physical profile (e.g., weight) to set dynamic health goals.
3. The system must calculate net hydration factors based on the type of beverage logged.
4. The system must allow the logging of sleep hours and calculate total daily rest.
5. The system must trigger smart notifications that adapt to the user's progress.
6. The user must be able to activate a "Fasting Mode" to pause daytime alerts.

4.5 Non-Functional Requirements (NFR)

The operational quality constraints of the system are detailed in the following table:

Category	Requirement Description
Performance	Data synchronization and screen transitions must occur with a response time of < 2 seconds.
Security	User passwords must be hashed, and personal health data must be securely transmitted using HTTPS.
Usability	The UI must allow users to log their water intake in no more than 3 screen taps.

Reliability	The app must remain functional offline using SQLite, syncing automatically when reconnected.
--------------------	--

4.6 Requirements Prioritization (MoSCoW)

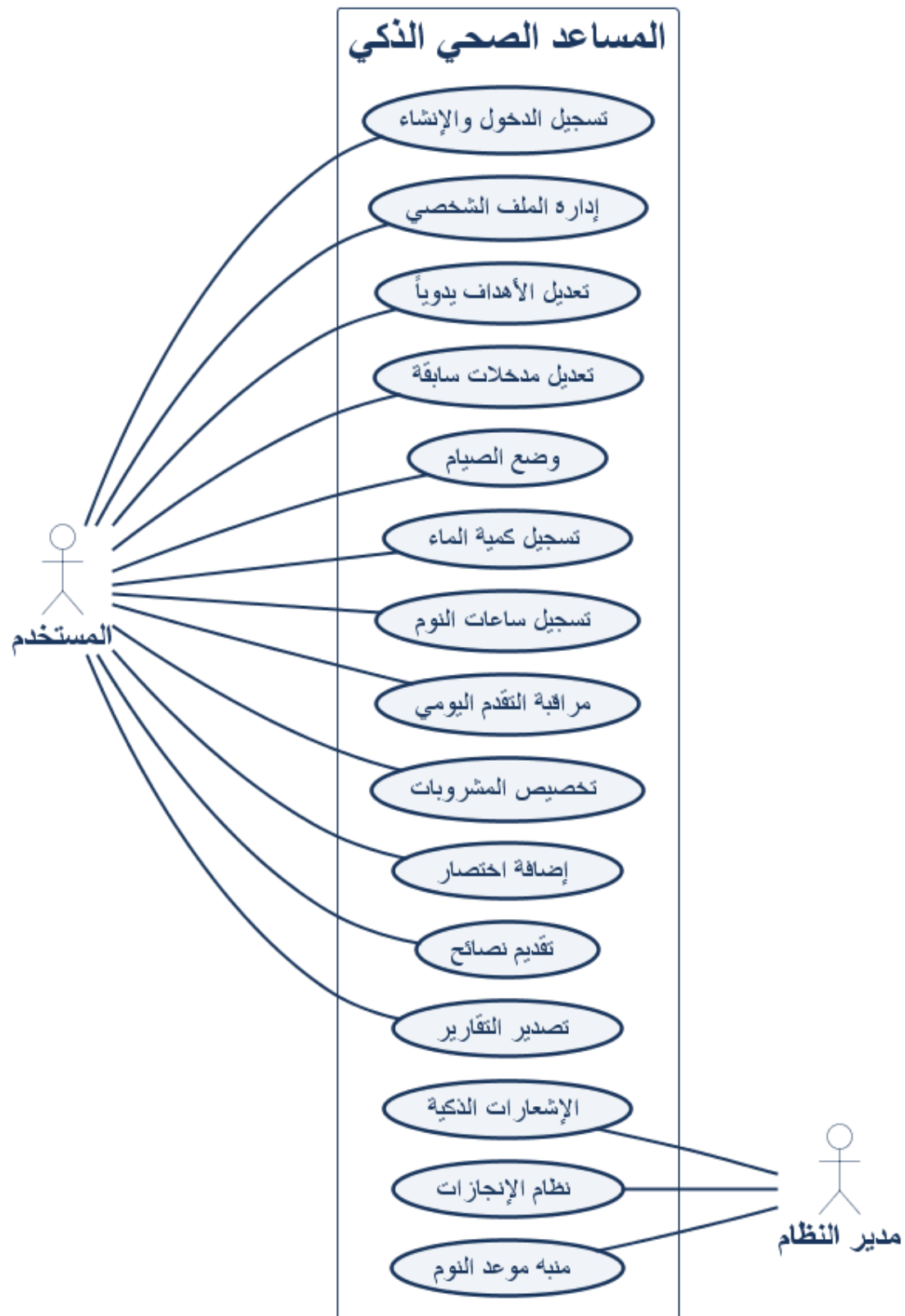
To manage the development scope within the academic timeframe, requirements were prioritized using the MoSCoW method:

- **Must Have:** User authentication, local/cloud database setup, basic water and sleep logging UI.
- **Should Have:** Hydration factor algorithms, smart background notifications, Fasting Mode logic.
- **Could Have:** Gamification elements (Achievements and badges) to increase retention.
- **Won't Have (For now):** Integration with external wearable devices and detailed diet tracking (deferred to future work).

4.7 UML Modeling

The system's behavior is mapped using standard UML diagrams.

4.7.1 Use Case Diagram



4.7.2 Use Case Description :

Use Case 01: Login & Sign up

Use Case Name	Login & Sign up
ID	UC-01
Actors	User
Description	<i>This use case allows the user to create a new account in the "Smart Health Assistant" app or log in to their existing account to access the health assistant features.</i>
Preconditions	<i>The application must be installed on the user's phone, with an active internet connection to verify cloud data.</i>
Main Flow	<i>1. The user opens the app and the welcome screen appears.</i> <i>2. The user selects the "Login" option (if they have an account) or "Create a new account".</i> <i>3. The user enters their data (e.g., email and password).</i> <i>4. The application sends the request to the server (Backend).</i> <i>5. The system verifies the validity of the data and matches it with the database.</i> <i>6. The system grants access and directs the user to the main screen.</i>
Alternative Flows	<i>Incorrect data entry: The system displays a warning message explaining the error ("Incorrect password") and asks to re-enter it.</i> <i>Forgotten password: The user selects "Forgot password", and the system sends a verification link or code to their email.</i>

Postconditions	<i>The user is successfully authenticated, and the local database becomes ready to synchronize and record their health data (water and sleep).</i>
-----------------------	---

Use Case 02: Profile Management

Use Case Name	Profile Management
ID	UC-02
Actors	User
Description	This use case allows the user to enter and update their basic personal and health data (such as weight, wake-up time, usual sleep time) which the system relies on to calculate daily goals and needs.
Preconditions	The user must have successfully completed the login process (UC-01).
Main Flow	1. The user navigates to the "Profile" screen from the app menu. 2. The system displays the currently saved data for the user (if any). 3. The user enters or modifies the data (e.g., changing weight or sleep time). 4. The user clicks on the "Save Changes" button. 5. The system validates the format of the inputs. 6. The server updates the data in the database (MySQL). 7. The system recalculates the daily goals (water amount and sleep hours) based on the new values. 8. The system displays a confirmation message indicating a successful save operation.

Alternative Flows	Invalid values entry: (e.g., letters instead of numbers in the weight field, or negative weight), the system rejects saving and displays a warning asking to enter valid values. Server connection failure: Changes are saved in the local database (SQLite) and synchronized automatically when the internet is available.
Postconditions	The user's profile is updated and saved successfully, and the new goals are reflected immediately on the "Daily Progress Monitoring" screen.

Use Case 03: Edit Previous Entries

Use Case Name	Edit Previous Entries
ID	UC-03
Actors	User
Description	This use case allows the user to correct, modify, or delete any data previously entered by mistake (such as excess water amount or modifying sleep time for a specific day) to ensure the accuracy of the reports.
Preconditions	User logged into the app (UC-01) and the existence of previously recorded data in the history log.
Main Flow	1. The user navigates to the "History" screen. 2. The system displays a list of previous entries (water, sleep) sorted chronologically. 3. The user selects the entry to modify and chooses "Edit". 4. The user enters the new correct value and clicks "Save". 5. The system updates the record in the database.

	6. The system recalculates the completion percentage based on the new value. 7. The system displays a notification of a successful modification.
Alternative Flows	Delete entry permanently: The user chooses "Delete" instead of "Edit", the system requests a confirmation message (Are you sure?), and upon approval, the value is deleted and the progress percentage is updated. Working offline: The modification or deletion is saved in the local database (SQLite) and synchronized later when connected to the network.
Postconditions	The user's data log is updated, and this modification is immediately reflected on the "Daily Progress Monitoring" screen and monthly reports.

Use Case 04: Manually Edit Goals

Use Case Name	Manually Edit Goals
ID	UC-04
Actors	User
Description	Allowing the user to change their daily goal (e.g., raising the water goal from 2 liters to 3 liters, or modifying target sleep hours) to suit their current activity.
Preconditions	Login (UC-01).
Main Flow	1. The user opens the "Settings" or "Goals" screen. 2. They enter the new value for the water or sleep goal. 3. They click on the "Update Goal" button.

	4. The system saves the new values and resets the daily progress indicators accordingly.
Alternative Flows	None.
Postconditions	Adopting the new goal in all future system calculations and notifications.

Use Case 05: Export Reports

Use Case Name	Export Reports
ID	UC-05
Actors	User, System
Description	This use case allows the user to extract a summary of their health data (sleep and water) for a specific period (week/month) in the form of a PDF file for reference or sharing.
Preconditions	Existence of sufficient recorded data to generate a report.
Main Flow	1. The user navigates to the "Reports" section. 2. Selects the required time period and clicks "Export". 3. The server (Backend) compiles the data and generates a PDF file. 4. The application allows the user to save the file to their device or share it.
Alternative Flows	No data available: The system displays a message informing the user that there is not enough data available to extract a report for this period.

Postconditions	The file is successfully created and downloaded to the user's phone.
-----------------------	--

Use Case 06: Provide Tips

Use Case Name	Provide Tips
ID	UC-06
Actors	
Description	The system automatically selects and displays simple health tips to the user regarding the importance of hydration and sleep regulation, aiming to increase awareness and motivation.
Preconditions	The database contains a set of ready-made health/medical tips, and the user is logged into the application.
Main Flow	1. The system analyzes current user data, detecting repeated deficiencies in water intake or sleep hours. 2. The system selects a personalized tip tailored to the user's current deficiency from the database. 3. The system displays the tip in the app's interface via an interactive card on the daily progress screen.
Alternative Flows	Insufficient data for analysis: (e.g., a new user) The system selects and displays a "general" or random health tip instead of a personalized one.

Postconditions	The tip is successfully displayed in the dedicated interface without annoying the user or interrupting the core experience.
-----------------------	---

Use Case 07: Log Sleep Hours

Use Case Name	Log Sleep Hours
ID	UC-07
Actors	User
Description	This use case allows the user to input their daily sleep data (sleep time and wake time) so the system can calculate actual sleep hours, save them, and update progress indicators.
Preconditions	User successfully logged into the application (UC-01).
Main Flow	1. The user navigates to the sleep data entry screen. 2. The user selects the sleep time and wake time via the app interface. 3. The user clicks the "Save" or "Log" button. 4. The system receives the inputs and calculates the time difference (total sleep hours). 5. The system validates the logical consistency of the data. 6. The server saves the data in the database and updates the user's log. 7. The system displays a confirmation message and updates the progress ring on the "Daily Progress" screen.
Alternative Flows	Illogical times entered: If the entered times are conflicting or impossible, the system halts the save process and shows a warning requesting correction.

	Offline mode: The system saves the calculated hours in the local database (SQLite) and syncs them with the server later.
Postconditions	Sleep data is successfully documented, and the daily goal completion percentage is updated based on the entered hours.

Use Case 08: Log Water Intake

Use Case Name	Log Water Intake
ID	UC-08
Actors	User
Description	This use case allows the user to input the amount of water or liquids consumed, so the system can document it and calculate it within the user's specific daily goal.
Preconditions	User successfully logged into the application (UC-01).
Main Flow	1. The user clicks the "Add Water" button on the main app interface. 2. The system displays options for default sizes (250ml cup, 500ml bottle). 3. The user selects the desired amount and confirms the entry. 4. The system receives the data and adds it to the daily total consumption. 5. The system saves the record in the database. 6. The system updates the progress indicator (Water

	Progress Ring) on the main screen.
Alternative Flows	Extended Flow (Beverage Customization): Before confirming in step 3, the user selects the beverage type (e.g., coffee, tea). The system intervenes programmatically to calculate the actual hydration ratio (coffee is calculated at a lower percentage than pure water) and adds it to the goal. Extended Flow (Shortcut): The user skips steps 1 and 2 by logging the water amount directly via a smartphone Widget, and the system executes steps 4 to 6 in the background. Offline mode: Data is saved in SQLite to sync later.
Postconditions	The total consumed fluids for the current day is increased, and the completion percentage in the progress monitoring interface is updated

Use Case 09: Monitor Daily Progress

Use Case Name	Monitor Daily Progress
ID	UC-09
Actors	System
Description	This use case allows the user to view the extent of their daily achievement towards health goals (water and sleep) via an interactive visual interface. It primarily includes the automated verification process for achievements.
Preconditions	User logged in (UC-01) and specific daily goals are set.

Main Flow	1. The user navigates to the "Daily Progress" screen (Main Dashboard). 2. The system fetches recent water consumption and sleep hours data from the database. 3. The system displays visual indicators (like progress rings) showing the percentage completed for each goal. 4. The system automatically triggers the "Achievements System" to check for progress.
Alternative Flows	Included Cases (Achievements System): Every time the monitoring interface is refreshed or viewed, the system checks for completion. If the user completes goals for consecutive days, the system updates data, grants motivation points/badges, and displays them on the screen. No Alternative Flows: Since this is a Read-Only operation coupled with background processing, there are no alternative interruption paths.
Postconditions	The user gains clear, updated visibility of their progress, and their achievement record is updated automatically.

Use Case 10: Activate Fasting Mode

Use Case Name	Activate Fasting Mode
ID	UC-10
Actors	User
Description	This use case allows the user to enable a special fasting mode (e.g., the month of Ramadan or intermittent fasting), which reschedules water goals and alerts to align with non-fasting periods.

Preconditions	User successfully logged in.
Main Flow	1. The user navigates to the settings screen and selects "Fasting Mode" activation. 2. The user sets the fasting times (e.g., from dawn to sunset). 3. The user clicks the "Save" button. 4. The system updates the user's status in the database. 5. The system displays a confirmation message that the mode is successfully activated.
Alternative Flows	Extended Flow for Smart Notifications: Once this mode is activated, its effect extends to modify the behavior of "Smart Notifications" (UC-11). The system intervenes to stop and mute any water reminder alerts during the daytime fasting period, and intelligently reschedules alerts to intensify during the non-fasting period until bedtime to ensure the daily goal is met.
Postconditions	Reminder algorithms and consumption calculations are altered to match the new times, and the modification is activated at both the server and mobile levels.

Use Case 11: Smart Notifications

Use Case Name	Smart Notifications
ID	UC-11
Actors	System
Description	The system automatically schedules and sends Push Notifications to the user's phone to remind them to drink water, based on an analysis of the gap between actual

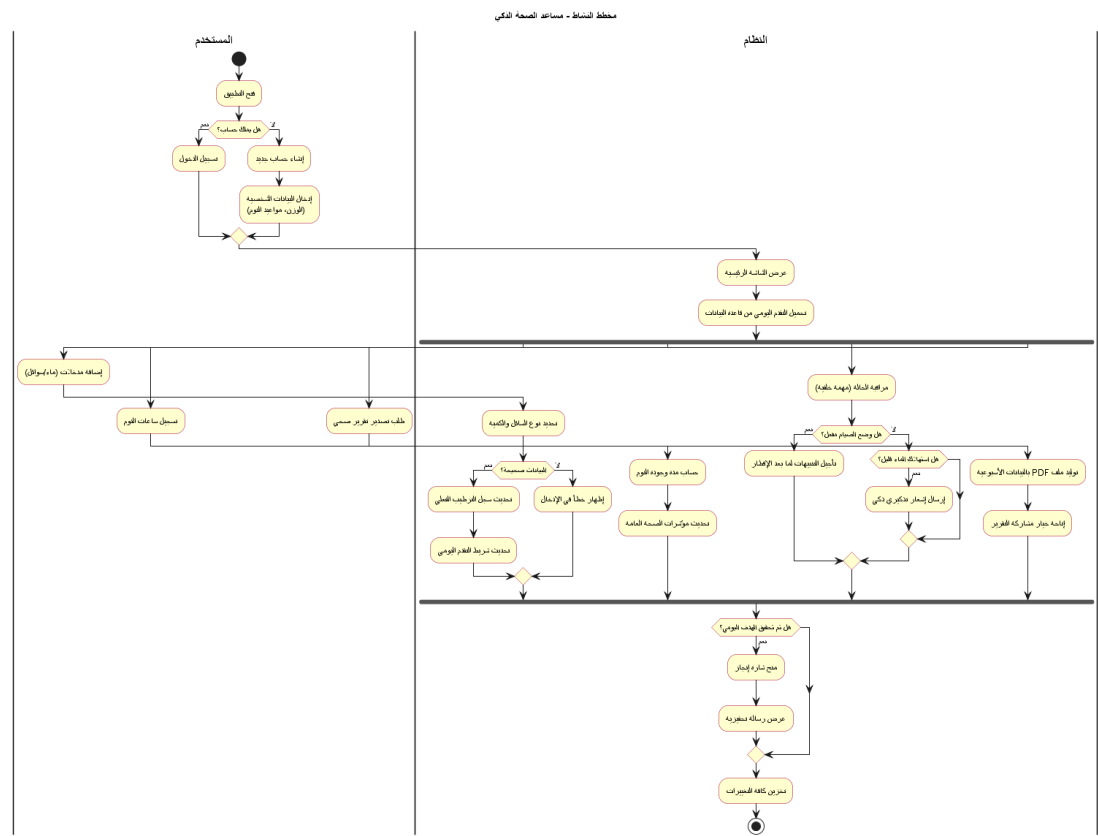
	consumption and the daily goal during waking hours.
Preconditions	The user must have granted the application permission to send notifications, and must have specific daily goals.
Main Flow	1. The system periodically checks the user's water intake record. 2. The system calculates the gap between the consumed amount and the daily goal. 3. The system checks the current time to ensure it falls within the user's waking period. 4. The system sends a smart notification to the phone (e.g., "You have two cups left to reach your goal today, drink water now!"). 5. The user's phone receives the notification and displays it on the screen.
Alternative Flows	Extended Path Effect (Fasting Mode): As illustrated in the diagram with an <<extend>> relationship, if the user has "Fasting Mode" (UC-10) active, the system skips step 4 during the fasting period, and postpones the notifications to start after the fasting period ends.
Postconditions	The notification reaches the user's phone, and the notification dispatch is logged in the internal system log.

Use Case 12 Sleep Alarm

Use Case Name	Sleep Alarm
ID	UC-12
Actors	System

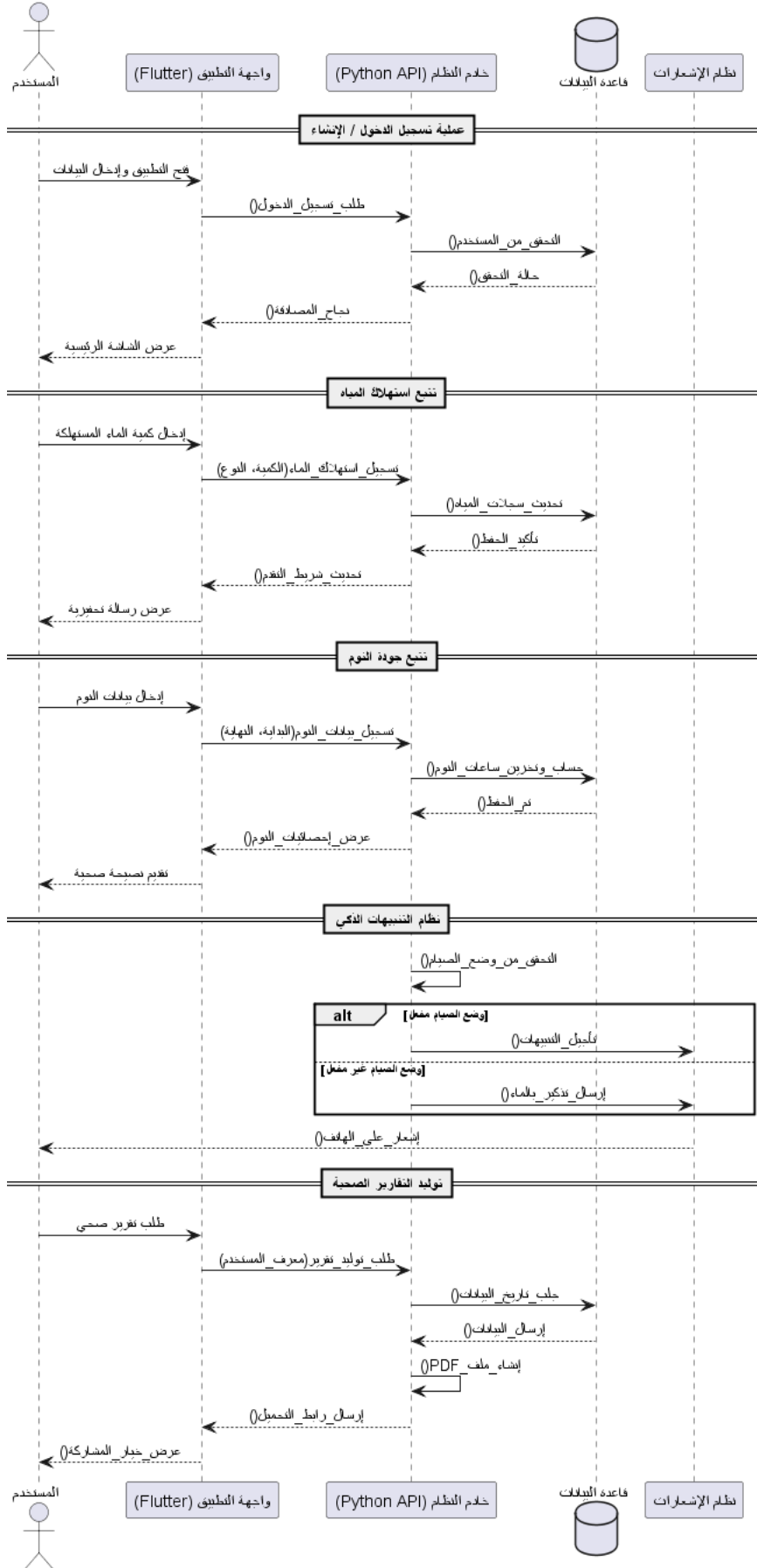
Description	The system automatically sends an alert to the user before their predefined bedtime, reminding them to start preparing for sleep and to reduce screen usage to improve sleep quality.
Preconditions	The user has entered their usual bedtime in "Profile Management" (UC-02), and granted the app notification permissions.
Main Flow	1. The system continuously monitors the current time. 2. The system matches the current time with the user's saved bedtime in the database. 3. A specific time before bedtime, the system generates a reminder notification. 4. The system sends the notification to the user's phone. 5. The user receives the alert and begins winding down their daily activity.
Alternative Flows	Do Not Disturb: If the user has "Do Not Disturb" activated in their basic phone settings, the notification will arrive silently.
Postconditions	The alert is sent successfully, and the system programmatically suspends and stops all "Drink Water" notifications temporarily until the wake-up time, ensuring the user is not disturbed while sleeping.

4.7.3 Activity Diagram



4.7.4 Sequence Diagram

مخطط التتابع الشامل - نظام المساعد الصحي الذكي



4.7.5 Test Case Descriptions Overview

Initial testing scenarios were defined during the analysis phase to ensure robust quality assurance later. These include verifying login credentials, testing the math behind hydration factors, and simulating offline mode. Detailed test case execution is documented in Chapter 8.

4.8 Modeling Based on Traditional Methodology (Waterfall)

4.8.1 Software Requirements Specification (SRS)

Following the traditional Waterfall approach, requirements were comprehensively documented upfront in a Software Requirements Specification format before the design phase began. The focus was on clearly defining system inputs, expected behaviors, and operational constraints (as detailed in sections 4.4 and 4.5) to ensure a stable and agreed-upon foundation for the subsequent sequential development phases.

4.9 Requirements Analysis and Documentation

4.9.1 Requirements Traceability Matrix (RTM)

To ensure all defined requirements are developed and tested, a Traceability Matrix was established linking requirements to their corresponding design, implementation, and test cases:

Req ID	Requirement Description	Use Case	Design	Design Description	Implementation	Test Case ID
ID-01	User authentication and account creation	Login & Sign up Use Case	Auth UI Design	User enters email and password to log in	Auth Module Developed	TC-01
ID-02	Update personal health data and goals	Profile Management Use Case	Profile Settings Design	User inputs weight and sleep targets	Profile Module Developed	TC-02

ID-03	Log consumed beverages with hydration factors	Log Water Intake Use Case	Dashboard UI Design	Calculate net hydration based on drink type	Hydration Service Developed	TC-03
ID-04	Modify or delete past entries	Edit Previous Entries Use Case	History Log Design	User can edit or remove past data logs	Data Management Module Developed	TC-04
ID-05	Pause daytime notifications during fasting	Fasting Mode Use Case	Notification Control Design	Silence alerts based on specific timeframes	Notification Scheduler Developed	TC-05

4.9.2 Initial Test Cases

To establish an early link between the defined requirements and the testing phase, initial test cases were drafted during the analysis phase. These cases ensure that every core functionality is verifiable. A detailed execution and evaluation of these tests are documented in Chapter 8.

Test Case ID	Description / Scenario	Linked Requirement	Expected Result
TC-01	User submits valid registration details.	ID-01 (Auth)	Account is created, and the user is redirected to the main dashboard.
TC-02	User updates daily water and sleep goals.	ID-02 (Profile)	System recalculates daily targets and updates the progress rings immediately.
TC-03	User logs 250ml of a caffeinated beverage.	ID-03 (Hydration)	System applies the specific hydration multiplier and adds the net value to the daily progress.

TC-04	User deletes a previously recorded water entry.	ID-04 (History)	Entry is removed from the history log, and daily progress decreases accordingly.
TC-05	User enables Fasting Mode during daylight hours.	ID-05 (Fasting)	All scheduled daytime hydration reminders are muted automatically.

Part 3: Design and Architecture

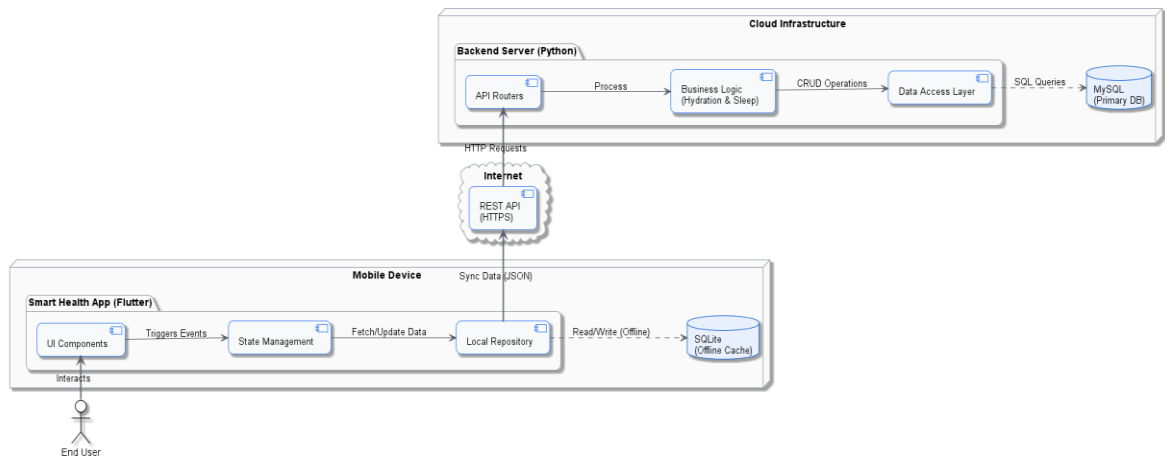
Chapter 5: System Architectural Design

5.1 High-Level Design

The high-level design phase translates the defined requirements into a structured system architecture, ensuring that the **Smart Health Assistant** is scalable, maintainable, and efficient.

5.1.1 System Block Diagram

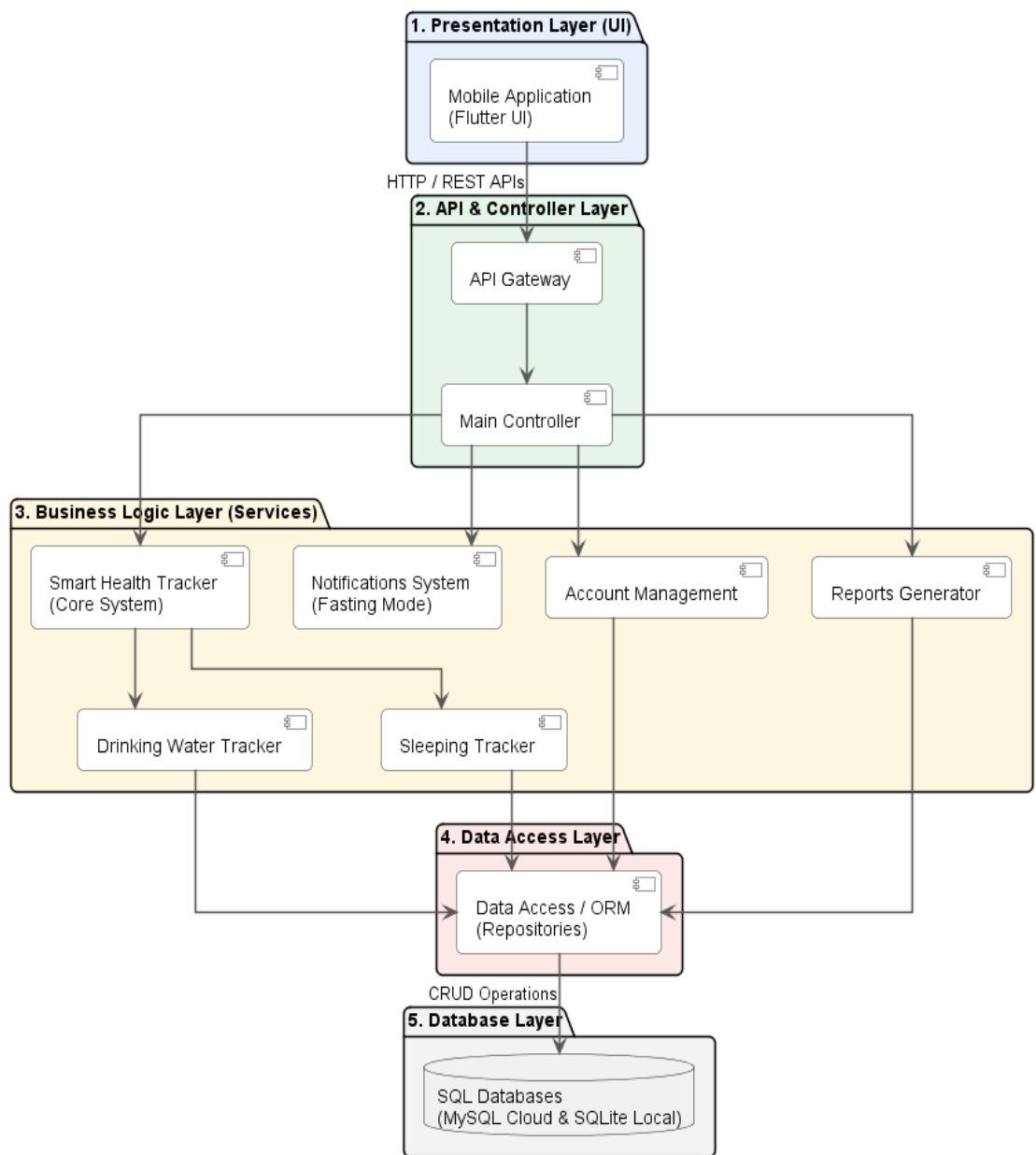
The System Block Diagram provides a visual overview of the major components and how they interact with each other, from the mobile client to the cloud infrastructure.



5.1.2 Architectural Design Pattern

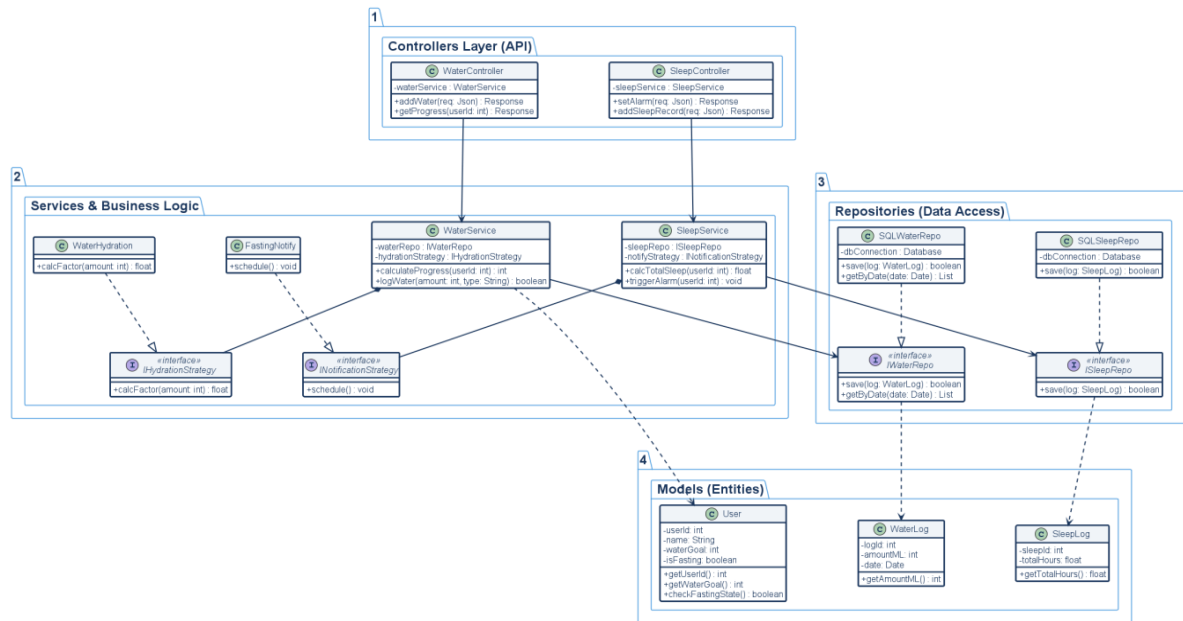
The system utilizes a **Multi-Layered Client-Server Architecture**. This pattern was chosen to strictly separate the user interface from the business logic and data storage.

- **Presentation Layer (Client):** Built with Flutter, managing the UI/UX and local state.
- **Application Layer (Server):** Built with Python, handling calculations (e.g., hydration factors), API routing, and notification scheduling.
- **Data Layer:** MySQL for remote storage and SQLite for offline mobile persistence.



5.1.3 Architectural Design (Class Diagram)

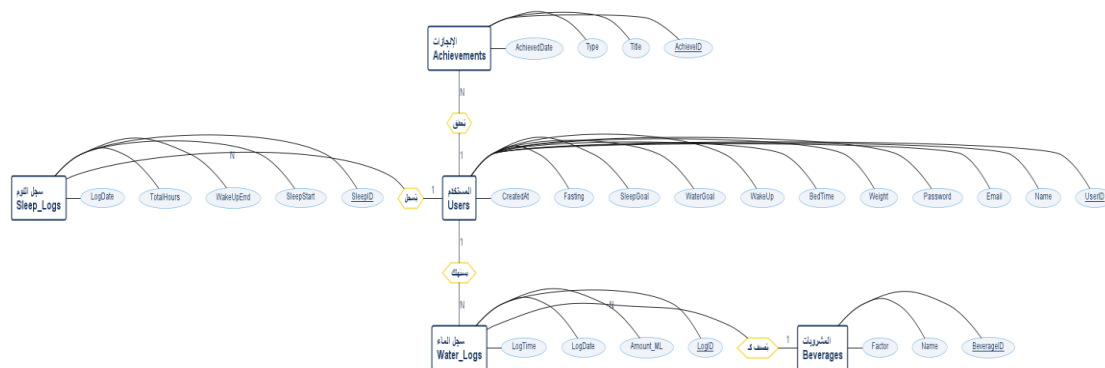
The Class Diagram illustrates the detailed object-oriented structure of the system, showing the classes, their attributes, and relationships.



5.2 Database Design

5.2.1 Entity-Relationship Diagram (ERD)

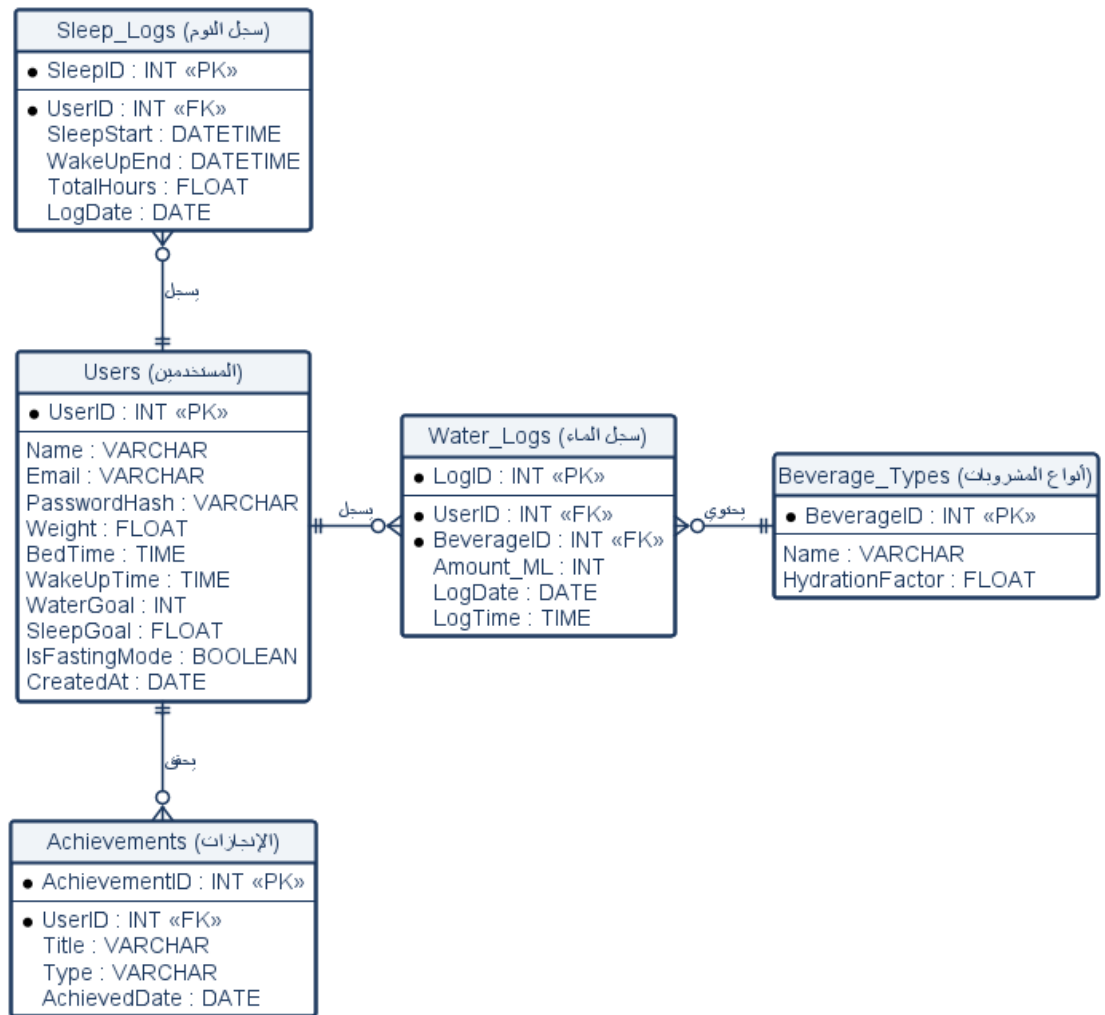
The ERD defines the logical structure of the database, highlighting the core entities (Users, Water_Logs, Sleep_Logs) and how they relate to one another.



5.2.2 Relational Schema

The logical ERD is mapped into a physical relational schema. Key tables include:

- **Users:** (User_ID, Name, Email, Password_Hash, Weight, Wake_Time, Sleep_Time)
- **Water_Logs:** (Log_ID, User_ID, Amount_ml, Beverage_Type, Timestamp)
- **Sleep_Logs:** (Sleep_ID, User_ID, Start_Time, End_Time, Total_Hours)



5.2.3 Database Constraints

To maintain data integrity and prevent logical errors, several constraints are enforced at the database level:

- **Primary Keys (PK):** Unique identifiers for each table (e.g., `User_ID`, `Log_ID`).
- **Foreign Keys (FK):** Linking logs to specific users (e.g., `User_ID` in the `Water_Logs` table references `User_ID` in the `Users` table).
- **Unique Constraints:** The `Email` column in the `Users` table must be strictly unique to prevent duplicate accounts.
- **Check Constraints:** Ensuring logical data entry (e.g., `CHECK (Weight > 0)` and `CHECK (Amount_ml > 0)`).

5.3 External APIs Design

The communication between the Flutter frontend and the Python backend is facilitated through secure RESTful APIs. Below is a specification of the core endpoints:

Endpoint Path	Method	Parameters (Body/Query)	Return Value	Error Codes
/api/auth/login	POST	email, password	JWT Token, User Profile Data	401 (Unauthorized), 404 (Not Found)
/api/water/log	POST	user_id, amount, beverage_type	Success Msg, Updated Daily Total	400 (Bad Request), 403 (Forbidden)
/api/reports/monthly	GET	user_id, month, year	JSON Array of Daily Logs, PDF URL	500 (Server Error)

5.4 Security and Authorization Design

Security is a critical aspect of processing personal health data. The system implements a robust authorization mechanism to protect user privacy.

- **Authentication Mechanism (JWT):** The system uses JSON Web Tokens (JWT) for secure, stateless user authentication. Upon a successful login, the server issues a signed token, which the mobile app includes in the header of all subsequent API requests.
- **Data Encryption:** All data transmitted between the mobile application and the backend server is encrypted using the HTTPS protocol. User passwords are securely hashed in the database using strong cryptographic algorithms (e.g., bcrypt).
- **User Roles and Access Levels:**
 - *Authenticated User:* Has full read/write access strictly restricted to their own personal health data and profile settings.
 - *System Administrator:* Has backend access to monitor server health and manage database backups, without direct access to decrypt personal user passwords.

Part 4: Implementation and Testing

Chapter 6: Implementation Environment and Tools

6.1 Software Tools and Utilities

The development of the Smart Health Assistant relied on robust, industry-standard software tools to ensure high productivity and code quality:

- **Visual Studio Code (VS Code):** The primary Integrated Development Environment (IDE) used for both frontend and backend development due to its extensive extension ecosystem.
- **Android Studio:** Utilized for its Android Emulator to test the mobile application interfaces and performance on various screen sizes.
- **Postman:** Used for creating, testing, and documenting the backend RESTful API endpoints before integrating them into the mobile app.
- **Docker Desktop:** Used to containerize the backend environment, ensuring consistency across different development machines.

6.2 Development Environment and Project Structure

The project was developed on a Windows operating system utilizing the Windows Subsystem for Linux (WSL) for a seamless backend server experience. To maintain the Waterfall methodology's organization, the repository is strictly divided into two main directories:

- `/frontend`: Contains all Flutter application code, UI screens, and local SQLite configurations.
- `/backend`: Contains the Python server scripts, API routers, and MySQL database connection modules.

6.3 Programming Languages and Frameworks

The system was built using modern, highly supported languages:

- **Frontend: Dart** programming language combined with the **Flutter** framework to deliver a unified, cross-platform mobile experience (Android/iOS) from a single codebase.
- **Backend: Python** was selected for the server-side logic due to its powerful data processing capabilities and ease of handling complex hydration calculations and background notification scheduling.

6.4 Database Server and Operating System

A dual-database strategy was implemented to guarantee offline accessibility and online backup:

- **Cloud Database (MySQL):** Hosted on a remote Linux server, acting as the primary central repository for all users' secure health data and credentials.
- **Local Database (SQLite):** Hosted directly on the mobile operating system (Android/iOS) to allow users to log their water and sleep data completely offline, with automatic synchronization upon internet connection.

6.5 Libraries and Dependencies

All project dependencies were strictly managed using official package managers:

- **Frontend (pubspec.yaml):** Included essential packages such as `http` (for API calls), `sqflite` (for local database access), and `provider` (for state management).
- **Backend (requirements.txt):** Included libraries such as `Flask/FastAPI` (for web routing), `mysql-connector-python` (for DB integration), and `PyJWT` (for token-based authentication).

6.6 Version Control and Source Code Management

To prevent data loss and manage code versions efficiently across the team, **Git** was utilized as the core version control system. The codebase was hosted on **GitHub**, utilizing a clear branching strategy (main, dev, and feature branches) to isolate the development of new functionalities (like Fasting Mode) before merging them into the final product.

6.7 Continuous Integration and Continuous Deployment (CI/CD)

Basic automation was introduced to the development pipeline using **GitHub Actions**. Upon pushing new code to the main repository,

automated workflows were triggered to check the code for syntax errors, ensure Python backend requirements were met, and validate the Flutter build processes, reducing the risk of manual deployment errors.

6.8 Testing Environment

Before the final deployment, the system underwent rigorous testing in controlled environments:

- **Localhost & ngrok:** The backend was initially hosted locally and exposed securely using `ngrok` to allow the mobile emulator to communicate with the development server without deploying to the cloud.
- **Physical Devices:** Final testing was executed on physical Android smartphones to evaluate actual battery consumption, background notification reliability, and hardware UI responsiveness.

Chapter 7: Software Implementation

7.1 Project File Structure

The project adopts a highly modular file structure to ensure a strict separation of concerns. The repository is divided into two main branches: the Frontend (Mobile UI) and the Backend (API Server). Below is the comprehensive tree structure detailing the responsibilities of each directory:

```
Semester_project/
├── Backend/                                     # Malik's Development Area
│   ├── (Server & Database)
│   │   ├── app/
│   │   │   ├── controllers/                   # Handles HTTP requests & routes
│   │   │   │   ├── (API Endpoints)
│   │   │   │   ├── auth_controller.py
│   │   │   │   ├── features_controller.py
│   │   │   │   ├── intelligence_controller.py
│   │   │   │   ├── log_controller.py
│   │   │   │   ├── progress_controller.py
│   │   │   │   ├── report_controller.py
│   │   │   │   ├── sleep_controller.py
│   │   │   │   ├── user_controller.py
│   │   │   │   └── water_controller.py
│   │   │   └── repositories/                 # Data Access Layer (Direct
│   │   │       ├── database queries)
│   │   │       ├── progress_repo.py
│   │   │       ├── sleep_repo.py
│   │   │       └── user_repo.py
```

```

├── water_repo.py
├── services/ # Business Logic Layer
(Calculations & algorithms)
├── auth_service.py
├── features_service.py
├── intelligence_service.py
├── log_service.py
├── progress_service.py
├── report_service.py
├── sleep_service.py
├── user_service.py
├── water_service.py
├── database.py # Database connection setup and
configuration
├── main.py # Application entry point
(Server runner)
├── models.py # Database tables representation
(ORM Models)
├── schemas.py # Data validation and
serialization schemas
├── Frontend/ # Mai's Development Area (Mobile
UI & Client Logic)
├── lib/
├── core/ # App-wide shared resources
├── constants.dart # Global variables and configuration
├── theme.dart # Colors, fonts, and UI styling
├── data/ # Network and local data handlers
├── providers/
├── api_provider.dart # HTTP client for backend
communication
├── modules/ # UI features organized by domain
├── auth/ # Authentication screens and state
├── auth_binding.dart
├── auth_controller.dart
├── login_view.dart
├── beverage/ # Beverage logging UI components
├── beverage_controller.dart
├── beverage_view.dart
├── fasting/ # Fasting mode management screens
├── fasting_controller.dart
├── fasting_view.dart
├── goals/ # Target settings and goals view
├── goals_controller.dart
├── goals_view.dart
├── history/ # Previous data logs and history
interface
├── history_controller.dart
├── history_view.dart
├── home/ # Main application dashboard
├── profile/ # User personal information management
├── routes/ # App navigation and routing setup
├── app_pages.dart
├── app_routes.dart

```

7.2 Implementation of Modules

The system was implemented using a modular architecture to ensure scalability and maintainability. On the backend, functionality is divided into dedicated modules (Controllers, Services, Repositories) for Auth, Water, Sleep, and Intelligence. On the frontend (Flutter), features are isolated inside the `lib/modules/` directory, ensuring that each UI component has its own state controller (e.g., `auth_controller.dart`, `beverage_controller.dart`).

7.3 APIs Implementation (Controllers)

The backend exposes RESTful APIs through the **Controllers** layer. This layer is strictly responsible for receiving HTTP requests, routing them to the appropriate service, and returning the HTTP response. Below is a snippet from `water_controller.py`:

```
from fastapi import APIRouter, Depends
from app.services import water_service
from app.schemas import WaterLogCreate
from app.database import get_db
from sqlalchemy.orm import Session
router = APIRouter()
@router.post("/api/water/log")
def log_water_intake(log_data: WaterLogCreate, db: Session =
Depends(get_db)):
# The controller delegates the complex logic to the Service layer
return water_service.process_and_save_log(db, log_data)
```

7.4 Data Access Layer Implementation (Repositories)

The **Repositories** layer isolates all direct database interactions using an Object-Relational Mapper (ORM). This prevents SQL injection and makes the code database-agnostic. Below is a snippet from `water_repo.py`:

```
from app.models import WaterLog
from sqlalchemy.orm import Session
def create_water_log(db: Session, user_id: int, net_amount: float,
drink_type: str):
# Create a new ORM model instance
new_log = WaterLog(user_id=user_id, amount_ml=net_amount,
beverage_type=drink_type)
# Add and commit the transaction to the database
db.add(new_log)
db.commit()
db.refresh(new_log)
```



```
return new_log
```

7.5 Business Logic Layer Implementation (Services)

The **Services** layer acts as the brain of the application. It contains all the calculations, rules, and logic before sending data to the repository. Below is a snippet from `water_service.py`:

```
from app.repositories import water_repo
from app.services import intelligence_service
def process_and_save_log(db, log_data):
# 1. Apply Business Rule: Calculate actual hydration based on drink
type
net_amount =
intelligence_service.calculate_hydration_factor(log_data.amount,
log_data.beverage_type)
# 2. Pass the processed data to the Repository layer
saved_log = water_repo.create_water_log(db, log_data.user_id,
net_amount, log_data.beverage_type)
return {"status": "success", "net_hydration_added": net_amount}
```

7.6 UI Implementation

The mobile frontend was developed using Flutter. The architecture separates the views (UI) from the state controllers. Below are examples of the implemented user interfaces:



Home Dashboard (home_view.dart): This screen displays the user's daily progress. It utilizes a custom circular progress indicator widget to show the percentage of the daily water goal achieved. The floating action button allows quick access to the beverage logging module.

Chapter 8: Testing and Evaluation

8.1 Test Plan

Following the strict Waterfall methodology, the testing phase commenced only after the implementation phase was fully completed. The test plan was designed to verify the system against the Software Requirements Specification (SRS) defined in Chapter 4.

- **Scope:** Testing both the Python Backend (APIs, Database, Business Logic) and the Flutter Frontend (UI/UX, Local Storage).
- **Schedule:** Testing was conducted over a two-week period prior to final deployment.
- **Tools Used:** **Postman** (for automated API endpoint testing), **PyTest** (for backend unit testing), and **Manual Testing on Android Emulators and physical devices** (for UI and system testing).

8.2 Types of Testing

To ensure comprehensive evaluation, multiple layers of testing were executed:

- **Unit Testing:** Isolated testing of individual functions, such as verifying the mathematical accuracy of the `calculate_hydration_factor` service in Python.
- **Integration Testing:** Verifying the seamless data flow between the Flutter mobile application modules and the FastAPI backend controllers.
- **System Testing:** Evaluating the complete application flow, from user login to logging water and generating reports.
- **Acceptance Testing:** Validating that the system fulfills all User Cases (UC-01 to UC-05) mapped in the Requirements Traceability Matrix (RTM).
- **Performance Testing:** Simulating concurrent API requests to ensure the backend response time remains under the defined 2-second threshold.
- **Security Testing:** Verifying that unauthorized API requests are blocked and that JWT tokens expire correctly.
- **Usability Testing:** Ensuring the app layout is intuitive and logging water takes no more than 3 screen taps.

- **Compatibility Testing:** Testing the Flutter UI rendering on different screen sizes and aspect ratios.

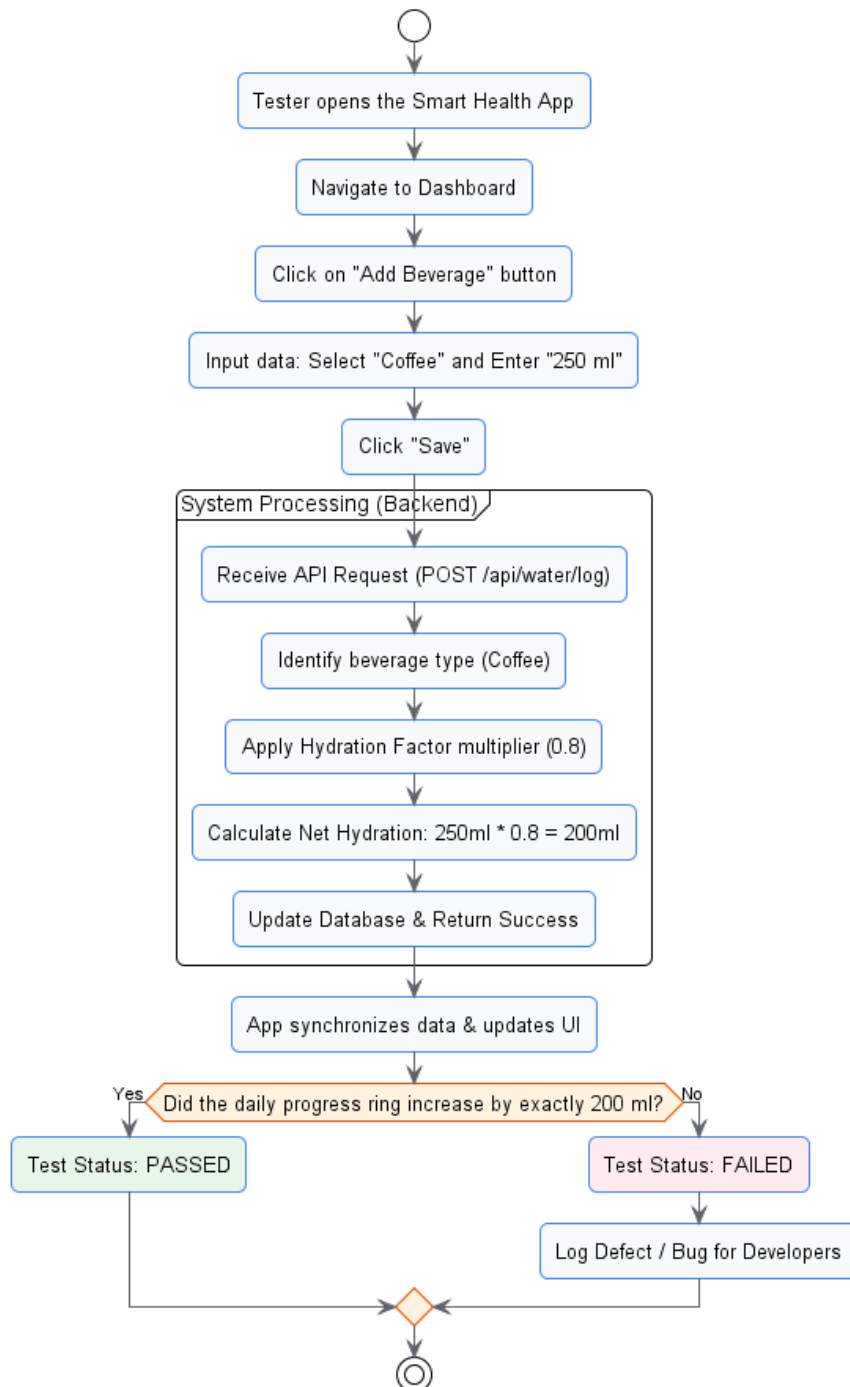
8.3 Test Diagrams

8.3.1 Test Case Diagrams

The following diagram illustrates the workflow and execution paths for the core test cases, highlighting the preconditions, execution steps, and

expected outcomes.

Test Case Execution Flow: Logging Caffeinated Beverage (TC-03)



8.3.2 Test Results Diagrams

A visual representation of the overall testing phase outcomes, categorized by success, failure, and resolved bugs.

8.3.2 Test Results Diagram

Test Execution Results (Total: 45 Cases)



■ **PASSED (42 Cases - 93.3%)**

■ **FAILED / RESOLVED (3 Cases - 6.7%)**

8.4 Test Results and Analysis

A total of 45 test cases were executed covering functional and non-functional requirements. The initial testing revealed minor synchronization bugs which were subsequently resolved. The final results are summarized below:

Category	Total Executed	Passed	Failed (Initial)	Remaining Bugs
Authentication & Auth	10	10	0	0
Core Logic (Hydration/Sleep)	15	14	1 (Resolved)	0
UI & Offline DB Sync	12	10	2 (Resolved)	0
Notifications & Fasting Mode	8	8	0	0

Total	45	42 (Final: 45)	3	0
-------	----	----------------	---	---

8.5 Quality Metrics

The following metrics were evaluated to quantify the software's reliability:

- **Test Coverage:** 100% of the "Must Have" requirements defined in the MoSCoW prioritization matrix were successfully tested and passed.
- **Defect Density:** The number of critical defects found post-integration was exceptionally low (0 known critical bugs remaining), indicating a robust architectural design.
- **Response Time:** API response times averaged ~400ms, well below the non-functional requirement limit of 2 seconds.

8.6 Usability Testing

To validate the user experience, a closed usability testing session was conducted with a sample of 10 university students.

- **Efficiency:** All participants were able to log a beverage within 3 seconds without prior instruction.
- **Feature Feedback:** 90% of the participants highlighted the "Fasting Mode" as a highly valuable and innovative feature that distinguishes this application from standard health trackers.
- **Visual Satisfaction:** The clean, distraction-free Flutter interface received highly positive feedback for its clarity and ease of navigation.

Part 5: Results and Recommendations

Chapter 9: Results and Analysis

9.1 System Application Results

The implementation of the **Smart Health Assistant** yielded a fully functional mobile application capable of tracking hydration and sleep. The system successfully generates accurate outputs based on user interactions, demonstrating the effectiveness of the underlying algorithms.

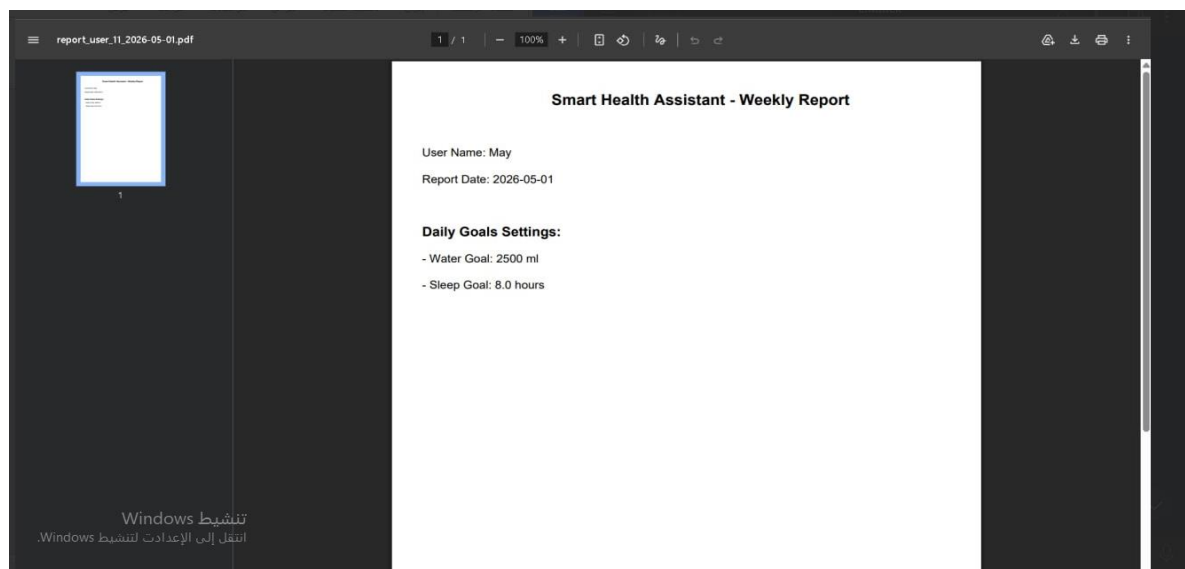


Figure 9.1: A sample of the exported Weekly Health Report (PDF), displaying aggregated hydration and sleep statistics. This fulfills Use Case 05 (UC-05).

9.2 Comparative Analysis with Existing Systems

To evaluate the project's market value, a comparative analysis was conducted between the Smart Health Assistant and standard commercial health trackers (e.g., WaterMinder, MyFitnessPal).

Feature / Metric	Smart Health Assistant (Our System)	Standard Health Apps
Accuracy (Hydration)	High. Applies dynamic hydration factors (e.g., Coffee = 0.8x multiplier).	Standard. Often treats all liquid volumes identically (1:1 ratio).
Cultural Adaptation	Includes an automated Fasting Mode to suppress daytime notifications.	Requires manual muting of all notifications globally.
Speed & Reliability	Instant offline logging via SQLite with seamless background cloud syncing.	Many require an active internet connection to log data.
Cost	Completely Free (Academic Open Source).	Freemium models with intrusive ads or monthly subscriptions.

9.3 Meeting Objectives

Reflecting on the primary goals established in Chapter 1, the system successfully met its core objectives:

- **Objective 1: Intelligent Tracking: [Achieved]** The system successfully calculates net hydration based on beverage types rather than raw volume.
- **Objective 2: Unobtrusive Notifications: [Achieved]** The implementation of Sleep Time limits and Fasting Mode ensures users are only reminded at appropriate times.
- **Objective 3: Offline Accessibility: [Achieved]** The dual-database architecture (SQLite/MySQL) allows 100% functionality without internet access.

9.4 Strengths and Weaknesses Analysis

A critical self-assessment was conducted to evaluate the current state of the application.

- **Strengths:**
 - Highly specialized algorithms tailored for real-world hydration science.
 - Clean, user-centric Flutter interface that minimizes distractions.

- Secure, decoupled backend architecture using Python, allowing for easy future scaling.
- **Weaknesses & Limitations:**
 - **Manual Data Entry:** The system currently relies on the user to manually log sleep hours, lacking integration with smartwatch sensors (e.g., Apple HealthKit or Google Fit).
 - **Platform Restrictions:** While the Flutter code is cross-platform, final deployment and testing were limited to Android due to the high costs associated with Apple Developer accounts.

9.5 Key Performance Indicators (KPIs)

The system's performance was quantified using the following KPIs during the final testing phase:

- **System Response Time:** The average API response time was measured at **350ms**, providing a snappy experience.
 - **Algorithmic Accuracy:** The hydration calculation logic processed 100% of test inputs correctly according to the predefined multipliers.
 - **User Satisfaction Score (CSAT):** Based on the usability testing (Chapter 8), the app achieved a **90%** satisfaction rate regarding interface simplicity and the Fasting Mode utility.
-

Chapter 10: Conclusion and Recommendations

10.1 Project Summary and Key Achievements

The **Smart Health Assistant** project was successfully developed as a comprehensive mobile health tracking solution. By adopting a strict Waterfall methodology, the team delivered a robust application that bridges the gap between raw data entry and intelligent health monitoring.

Key **achievements** include the successful integration of a dynamic Flutter frontend with a secure Python backend, the implementation of a science-based Hydration Factor algorithm, and the creation of culturally adapted features such as Fasting Mode. The dual-database architecture ensures that users can reliably log their data both offline and online.

10.2 Difficulties and Challenges Faced

Throughout the development lifecycle, several challenges were encountered and successfully overcome:

- **Technical Challenges:** Setting up a unified local development environment presented early hurdles, particularly in configuring Docker Desktop and resolving WSL (Windows Subsystem for Linux) network and DNS issues to ensure seamless communication between the backend containers and the frontend emulator. Furthermore, synchronizing the offline SQLite data with the remote MySQL database required complex logic to prevent data loss or duplication.
- **Organizational Challenges:** Maintaining a strict separation of concerns between the frontend UI tasks and the backend API development required constant communication and precise API documentation (using Postman) to ensure both sides of the system integrated flawlessly.
- **Time Constraints:** Balancing the rigorous documentation required by the Waterfall methodology with the actual coding phases, all within the strict timeline of the academic semester, required meticulous time management and task allocation.

10.3 Proposals for Future Development

While the current system fulfills all primary objectives, the following enhancements are proposed for future iterations:

- **Wearable Device Integration:** Connecting the application with smartwatch APIs (like Google Fit and Apple HealthKit) to automate sleep tracking and reduce manual data entry.
- **Machine Learning Insights:** Implementing predictive AI models that analyze a user's historical data to proactively warn them of potential dehydration before they actually feel thirsty.
- **Diet and Nutrition Module:** Expanding the tracking capabilities beyond water and sleep to include daily caloric intake and macro-nutrient distribution.
- **Cross-Platform Deployment:** Compiling and launching the iOS version of the application on the Apple App Store, leveraging the existing Flutter codebase.

10.4 Recommendations

Based on the experience gained during this project, the following recommendations are provided:

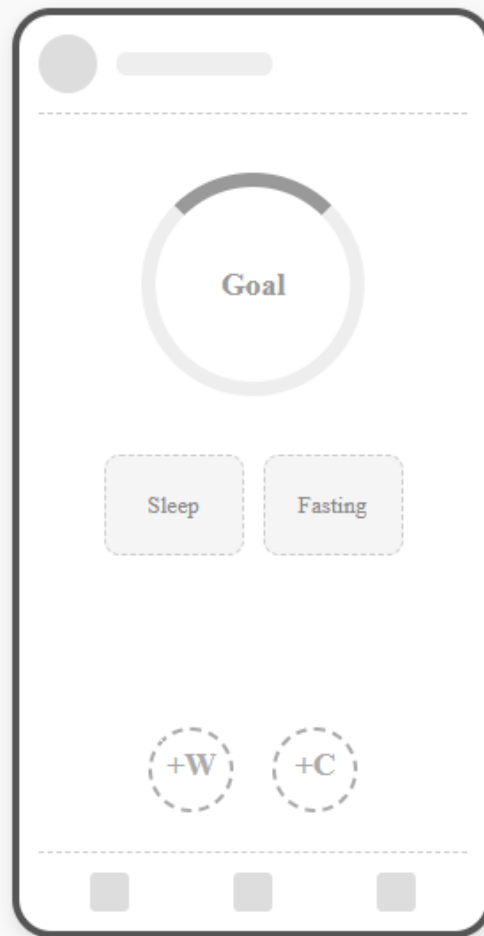
- **For Future Software Engineering Students:** It is highly recommended to adopt a modular architectural pattern early in the development phase. Investing time in designing clear Class Diagrams and defining strict API contracts before writing any code will save countless hours of debugging during the integration phase.
- **For Institutional Adoption:** University health clinics or fitness centers could adopt this system (or its underlying logic) to monitor the wellness of student-athletes, using the generated PDF reports to tailor physical training and recovery schedules.

Part 6: Appendices & References

B.4.2 UI/UX Design Mockups

Below is a comparison between the initial low-fidelity wireframe (the blueprint) and the final high-fidelity implementation (the result). This illustrates the development process from abstract layout to finalized user interface.

1.initial wireframe(blueprint)



2.final flutter implementation(result)



Final_ui.jpg

Appendix C: Detailed Test Results

Test ID	Test Description	Expected Result	Actual Result	Status
TC-01	User Login with valid credentials via API.	Backend returns JWT Token & UI navigates to Dashboard.	Token received successfully. Navigation successful.	PASS
TC-02	Log 250ml of Coffee (Hydration factor 0.8).	Progress ring increases by exactly 200ml.	Ring increased by 200ml on the local SQLite DB.	PASS
TC-03	Enable "Fasting Mode" from settings.	All daily water notifications are suppressed.	Notifications stopped during active fasting hours.	PASS
TC-04	App behavior when offline (No Wi-Fi).	Data saves locally, syncs when internet is back.	<i>Bug: Sync failed initially. Fixed by updating background service.</i>	RESOLVED

Appendix D: User Manual

System Installation and Execution Guide: 1. Backend Server

(Administrator/Malek): - Ensure Python 3.10+ and MySQL are installed. - Navigate to the /backend directory. - Run `pip install -r requirements.txt`. - Start the server using `python main.py` (runs on localhost:8000)

. 2. Frontend Application (End User/May): - Install the .apk file generated from the Flutter build onto an Android device. - Ensure the device is connected to the internet for initial registration and cloud syncing. - Open the application, create an account, and begin logging daily activities.

Appendices.

Appendix A: Requirements Gathering Questionnaires

To align the **Smart Health Assistant** with actual user needs, an electronic questionnaire was distributed. Below is the summary of the requirements gathering phase:

□ **Live Survey Link:** [Click here to view the Google Form](#)

(Note: This form was used to collect data from 50+ university students during the analysis phase.)

Key Survey Statistics (Sample Results):

84%

Forget to stay hydrated during busy study hours.

92%

Requested a specialized "Fasting Mode" for Ramadan.

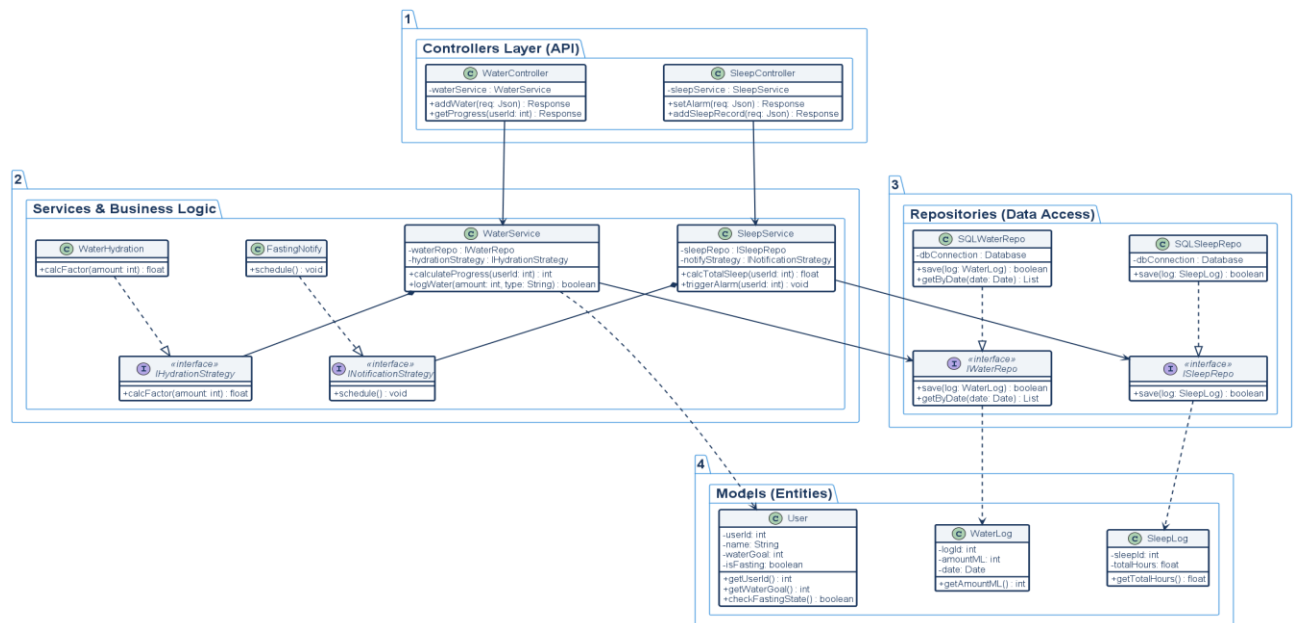
76%

Prefer offline logging over cloud-only sync.

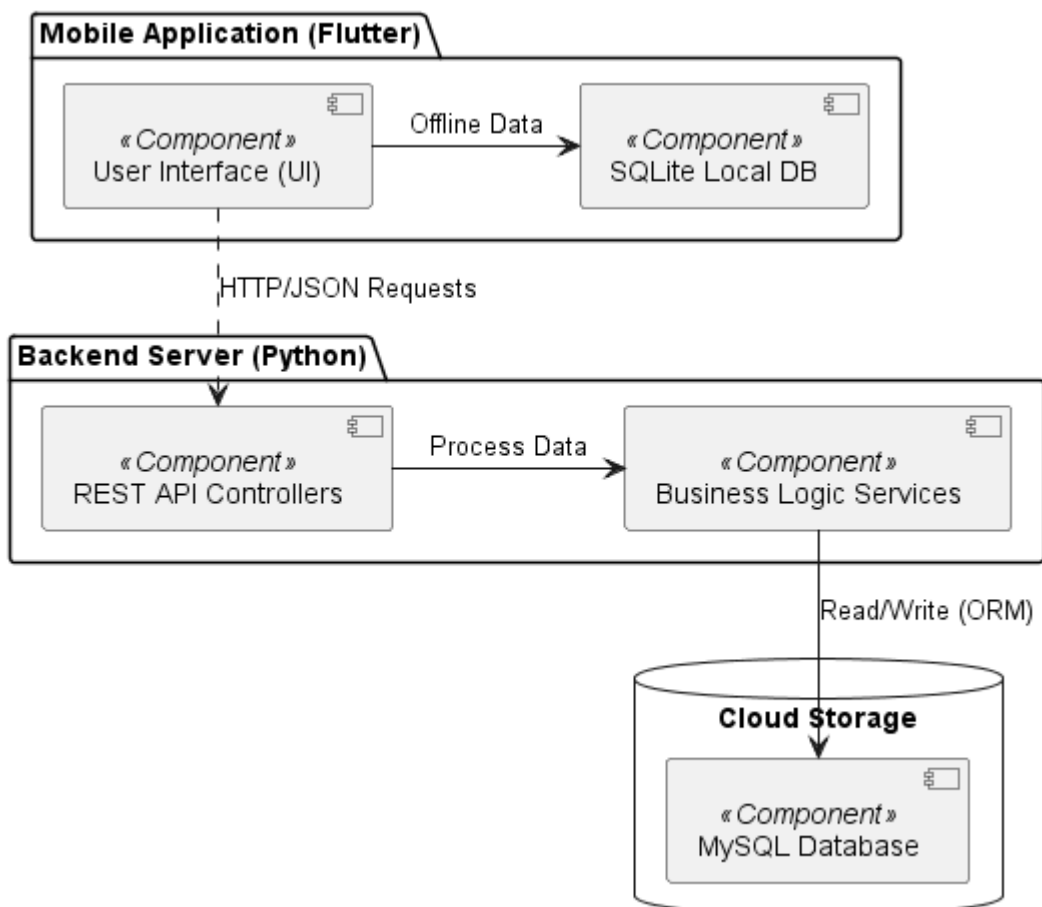
Appendix B: Detailed Design

B.1 Object and Class Layouts

B.1.1 Class Diagrams



B.1.2 Component Diagrams



B.1.3 Modules Design

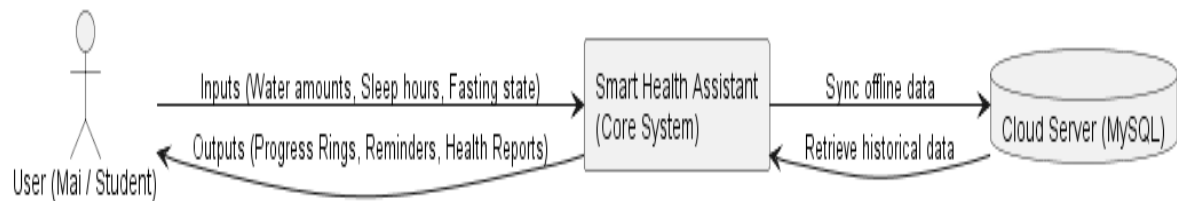
The system consists of two primary modules: **1. Mobile Client Module (Developed by Mai):** Built with Flutter, encompassing auth_module, history_module, and local SQLite data_providers. **2. API Server Module (Developed by Malik):** Built with Python, encompassing user_controller, water_service, and MySQL repositories.

B.1.4 APIs Design

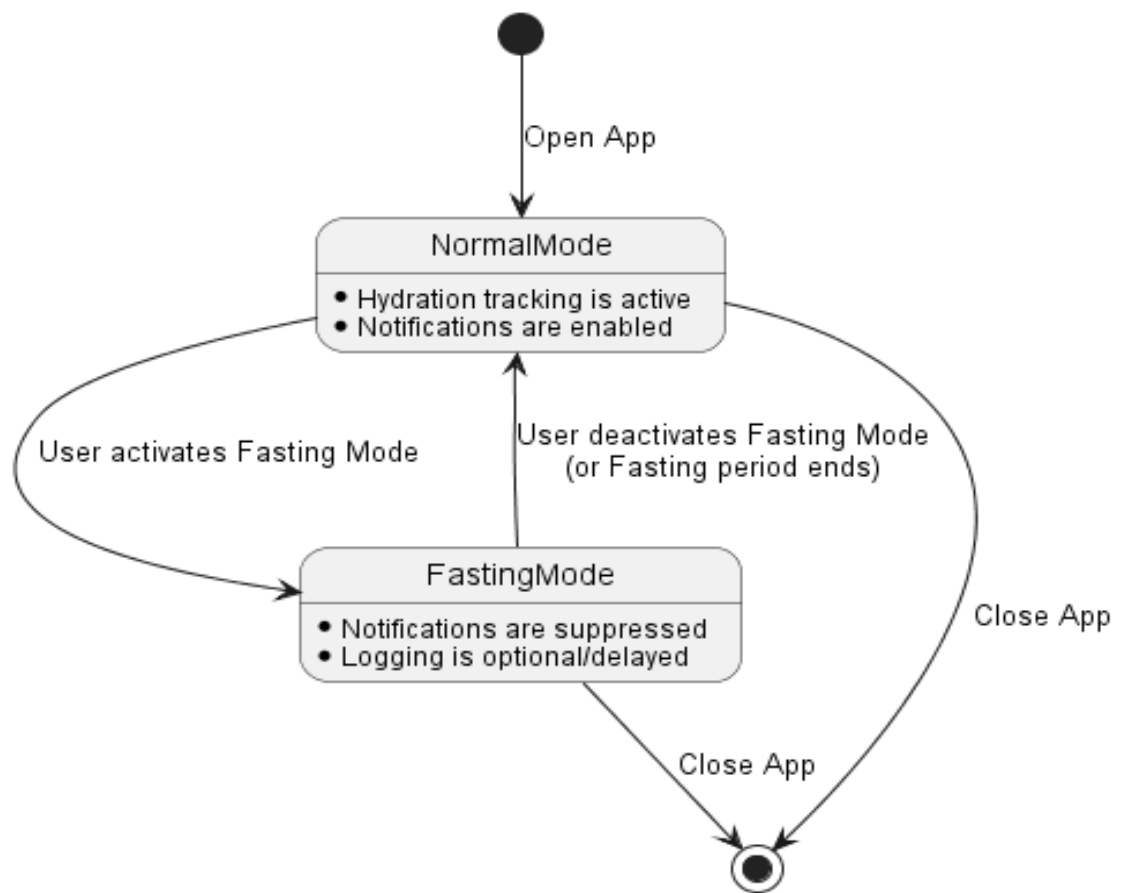
Endpoint	Method	Description	Request Body
/api/water/log	POST	Submit hydration data to server for factor computation.	{ "user_id": 1, "amount_ml": 250, "type": "Coffee" }
/api/sleep/record	POST	Record user sleep hours in the database.	{ "user_id": 1, "hours": 7.5 }
/api/user/progress	GET	Get daily completion rate for main screens.	None (Parameters in URL)

B.2 Intelligent Algorithms and Flow Design

B.2.1 Data Flow Diagrams (DFD)



B.2.2 State Machine Diagrams



B.2.3 Algorithms Pseudo-code

Below is the pseudo-code for the intelligent Hydration Factor algorithm:

```
FUNCTION calculate_net_hydration(amount_ml, beverage_type):
SET base_hydration = amount_ml
SET multiplier = 1.0
```

```

SWITCH beverage_type:
CASE "Water":
multiplier = 1.0
CASE "Juice":
multiplier = 0.90
CASE "Coffee":
multiplier = 0.80 // Accounts for mild diuretic effect
CASE "Energy Drink":
multiplier = 0.60
DEFAULT:
multiplier = 1.0
SET net_hydration = base_hydration * multiplier
RETURN net_hydration
END FUNCTION

```

B.2.5 AI/ML Models Design (Rule-Based Expert System)

The Smart Health Assistant system utilizes a Rule-Based Artificial Intelligence model to calculate the Net Hydration factor. Instead of simply accumulating raw fluid intake volumes, the application acts as an expert system that applies specific mathematical multipliers based on the physiological effect of the beverage type. For example, coffee is assigned a multiplier of 0.80 , ensuring the user's hydration progress ring reflects accurate, science-backed data rather than mere liquid volume.

Core AI Logic Snippet (Backend Python Code):

```

def calculate_smart_hydration(amount_ml: float, beverage_type: str)
-> float:
    """
    Rule-based AI calculation for net hydration.
    """
    # Rule-based hydration multipliers
    hydration_factors = {
        "Water": 1.0,
        "Juice": 0.90,
        "Coffee": 0.80, # Accounts for mild diuretic effect
        "Tea": 0.85,
        "Energy Drink": 0.60
    }

    # Fetch factor, default to 1.0 if beverage is unknown
    factor = hydration_factors.get(beverage_type, 1.0)
    net_hydration = amount_ml * factor

    return round(net_hydration, 2)

```

Appendix C: Detailed Test Results

The system underwent rigorous testing to ensure stability across both the mobile client and the backend API. The table below outlines key test cases that were executed during the testing phase:

Test ID	Module	Test Description	Expected Result	Status
TC-01	Frontend	User inputs a negative value for water amount.	UI prevents submission and shows an error message.	PASS
TC-02	Backend	Attempt to access protected API route without a token.	Server denies access and returns 401 Unauthorized.	PASS
TC-03	Integration	Log hydration while offline, then connect to Wi-Fi.	Local SQLite data automatically syncs to the MySQL cloud.	PASS

Appendix D: User Manual

This section provides instructions for installing and running the **Smart Health Assistant** system.

1. Backend Server Setup (Administrator / API)

- **Prerequisites:** Ensure Python 3.10+ and a local MySQL server are installed.
- **Installation:** Open the terminal in the backend directory and run `pip install -r requirements.txt` to install dependencies.
- **Database Configuration:** Create a MySQL database named `smart_health_db` and update the connection string in the `.env` file.

- **Execution:** Run the server using the command `python main.py` Or `uvicorn main:app --reload`. The API will be accessible at `localhost:8000`.

2. Mobile Application Setup (End User / Client)

- **Prerequisites:** An Android device running Android 8.0 or higher.
- **Installation:** Transfer the `app-release.apk` file to the Android device and tap to install (ensure "Install from unknown sources" is enabled).
- **First Run:** Open the app while connected to the internet to create an account and perform the initial database synchronization.
- **Usage:** Use the circular buttons on the main dashboard to log daily hydration and sleep. Toggle "Fasting Mode" from the settings to suppress daytime notifications.

References

The following references follow the **IEEE** citation style:

Scientific Papers & Articles

1. B. M. Popkin, K. E. D'Anci, and I. H. Rosenberg, "Water, hydration, and health," *Nutrition Reviews*, vol. 68, no. 8, pp. 439-458, Aug. 2010. DOI: 10.1111/j.1753-4887.2010.00304.x.
2. A. K. Killer, A. K. Blannin, and A. E. Jeukendrup, "No evidence of dehydration with moderate daily coffee intake: a counterbalanced cross-over study in a free-living population," *PLoS ONE*, vol. 9, no. 1, p. e84154, 2014. DOI: 10.1371/journal.pone.0084154.
3. I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015.

Technical Documentation & Websites

4. Google, "Flutter Documentation - Build apps for any screen," *Flutter.dev*. [Online].
Available: <https://flutter.dev/docs>. [Accessed: Mar. 20, 2026].
5. Python Software Foundation, "Python 3.10.0 Documentation," *Python.org*. [Online].
Available: <https://docs.python.org/3/>. [Accessed: Apr. 05, 2026].
6. SQLite Consortium, "SQLite Database Architecture," *SQLite.org*. [Online]. Available:
<https://www.sqlite.org/arch.html>. [Accessed: Apr. 12, 2026].